BASEMIND · REFERENCE

The BaseMind Field Manual

Everything about the app a Scout group runs on — what each screen does, how the pieces fit together, how to stand a copy up from nothing, and where the sharp edges are.

Commit 3cb1629 · master **Verified** 27 August 2026 **Screens** live app, sample data

01 What BaseMind is

BaseMind is a web app for the day-to-day administration of one Scout group. It answers the questions a quartermaster and a camp leader actually ask:

- What kit do we own, what condition is it in, and which room is it in?
- Who has borrowed what, and when is it due back?
- What camps are coming up, and where are they?
- What do we need to pack, what do we need to eat, and what do we still need to buy?
- Where is the nearest hospital to this site, and who is the first aider?
- What do our group's own written policies say about *this*?

It is a rebuild. The original attempt tried to do everything at once and collapsed; this version started as equipment tracking plus roles, on Supabase, with access enforced by Postgres Row Level Security, and has grown one working feature at a time.

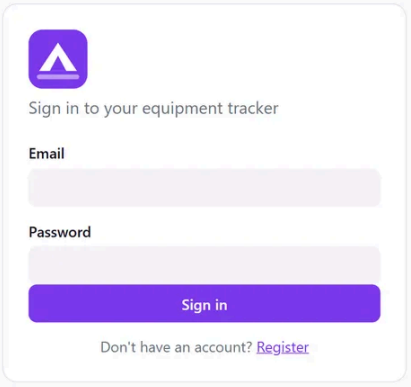
Design commitments

Four ideas run through the whole codebase, and most of the code makes more sense once you know them.

COMMITMENT	WHAT IT MEANS IN PRACTICE
The database is the authority	A Viewer cannot write, even by calling the Supabase REST API directly with a valid token. Hidden buttons are a convenience, not the control.
It has to work at camp	Tables are cached in IndexedDB, the shell is precached by a service worker, adds are queued offline and replayed, and the profile survives a dead connection so a leader isn't bounced to a login screen when the safety plan is needed.
A model's output is checked, not trusted	Both AI features verify the reply against the source data and discard it if it doesn't match. Every AI feature has a working non-AI path.
Nothing is invented	The safety plan renders only facts that were typed in. The assistant answers only from uploaded documents, and every quote is looked up in the document before it is displayed.

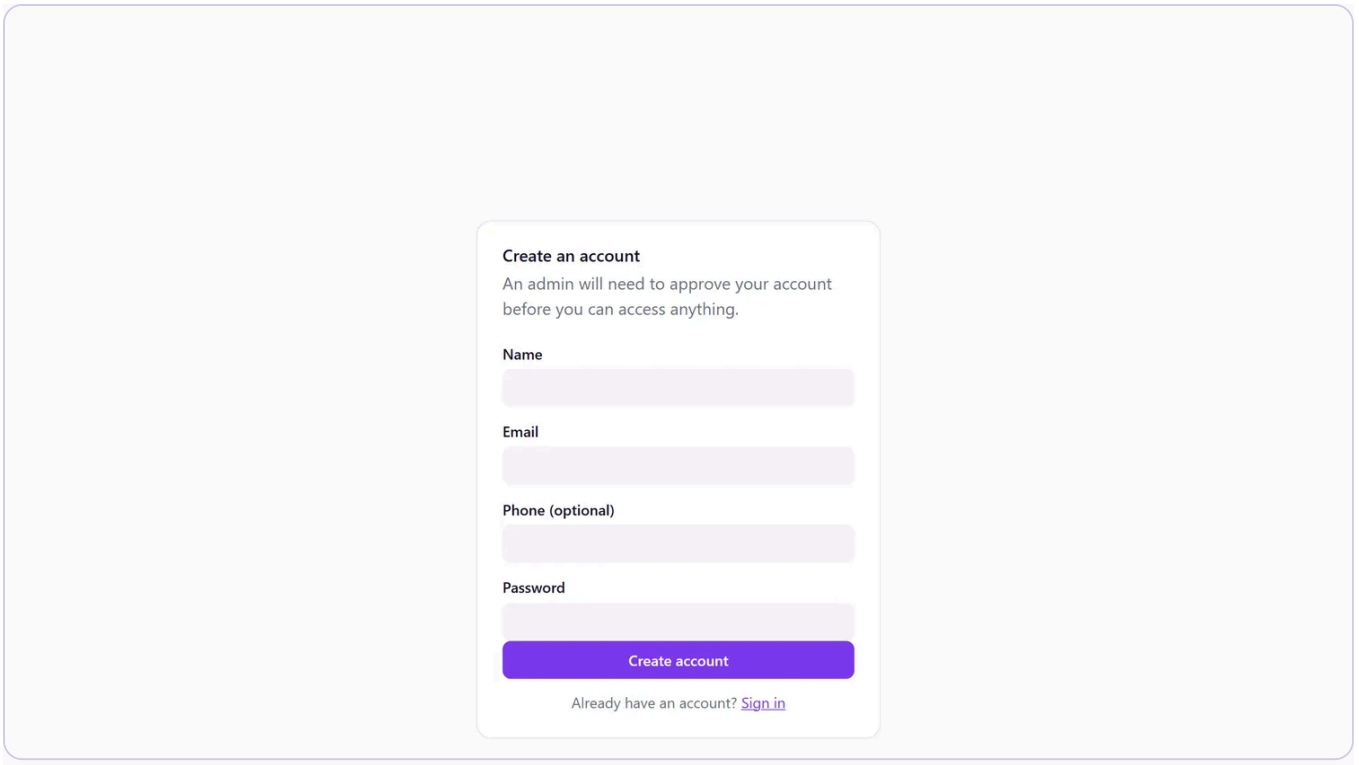
02 A tour of the app

Signing in



The sign-in screen. Two other public routes exist: `/register` and `/pending`.

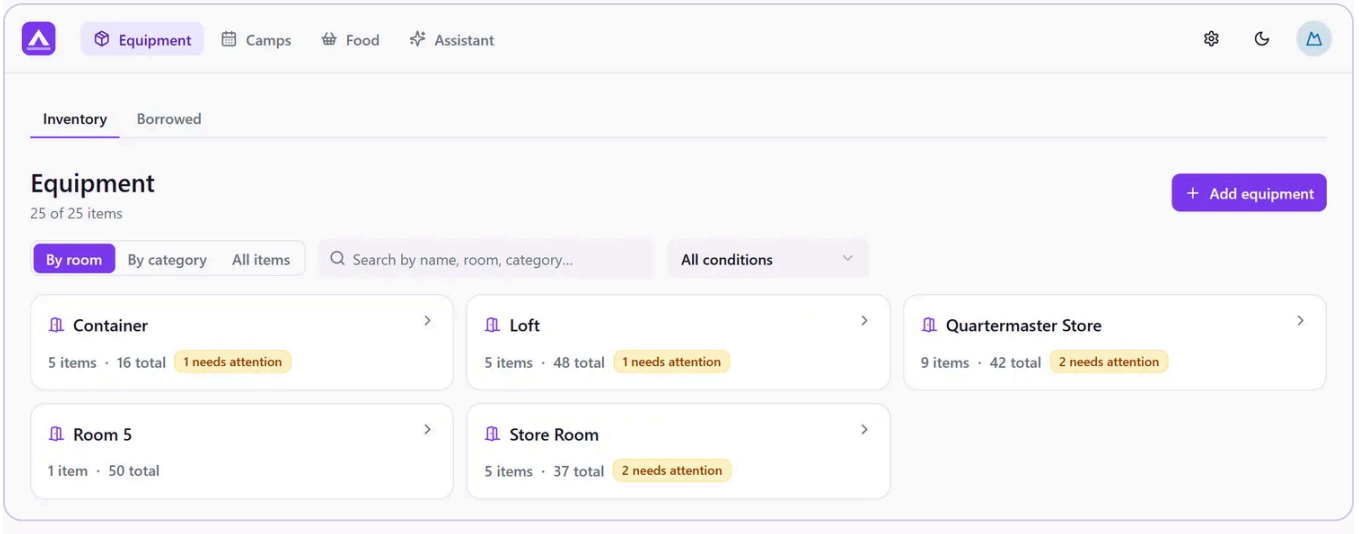
Registration always creates a **Pending / Viewer** account regardless of what the browser sends — role and status are hard-coded in a database trigger, so self-registration cannot grant itself access. A new account lands on the pending screen until an Admin approves it.



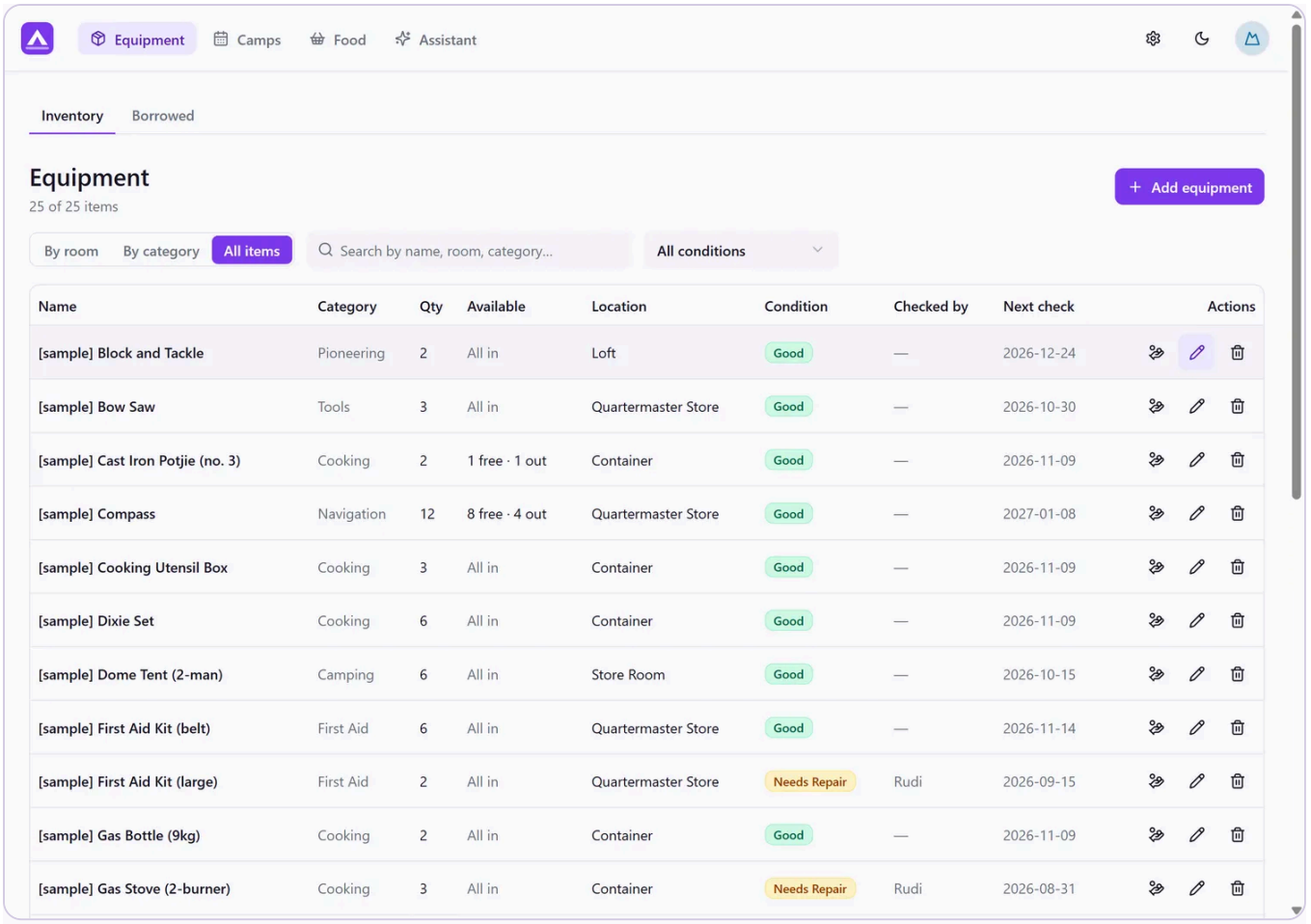
Registering. Only name and phone are taken from the client; role and status are not.

Equipment

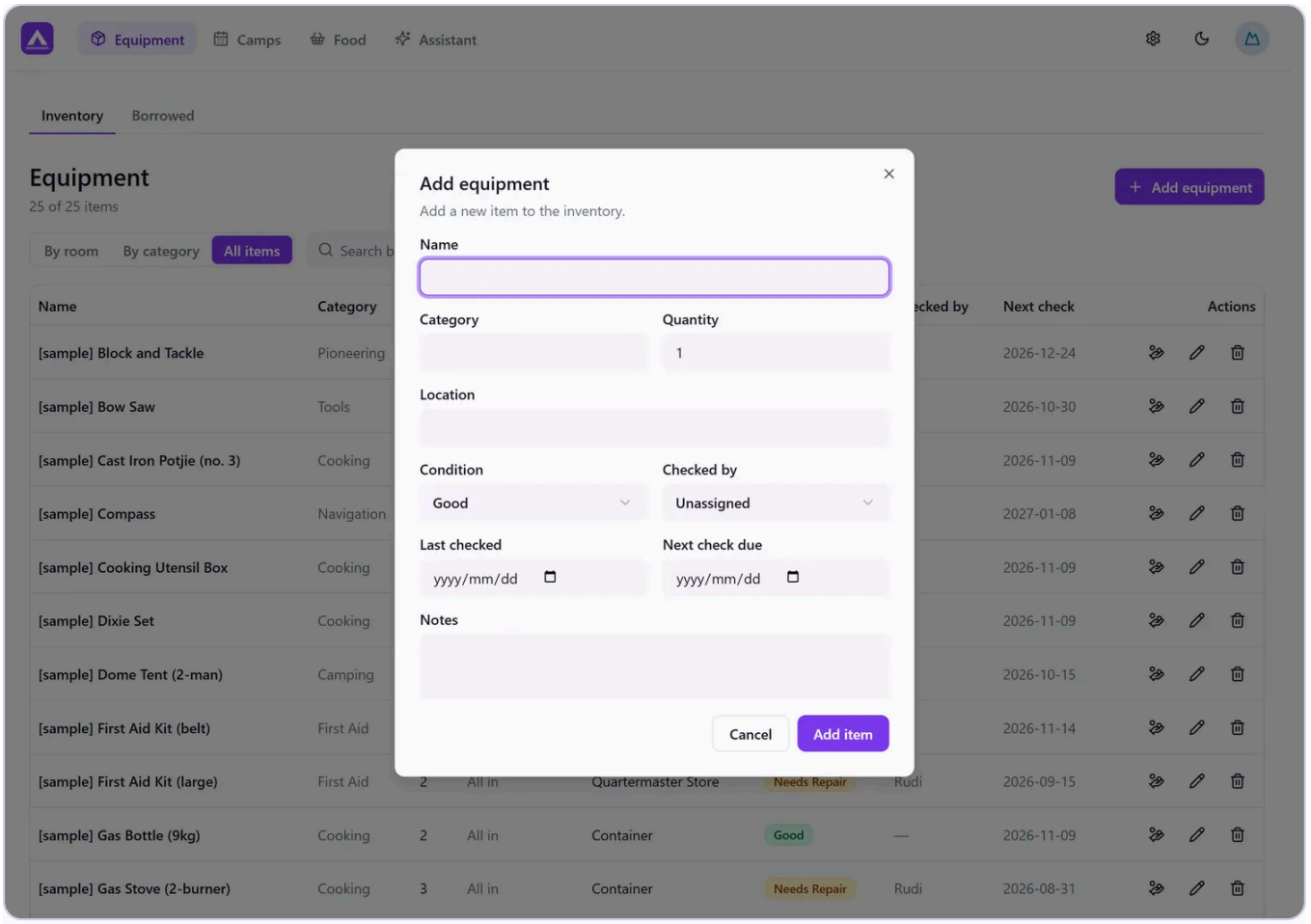
The default landing page. Items are grouped **by room**, with a count, a total quantity, and a badge for anything needing attention.



Equipment grouped by room. "By category" and "All items" are the other two views.



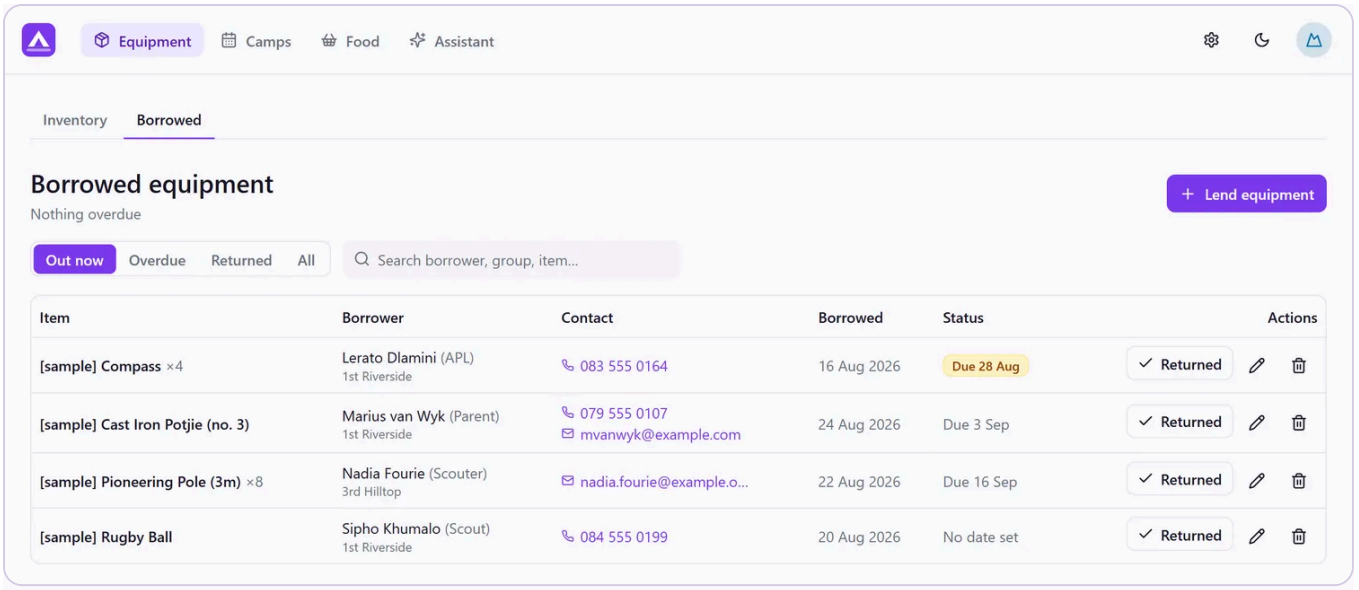
The flat table. Search matches name, room and category; the dropdown filters by condition.



Adding an item. `condition` is constrained by the database to Good, Needs Repair, Broken or Missing.

Borrowing

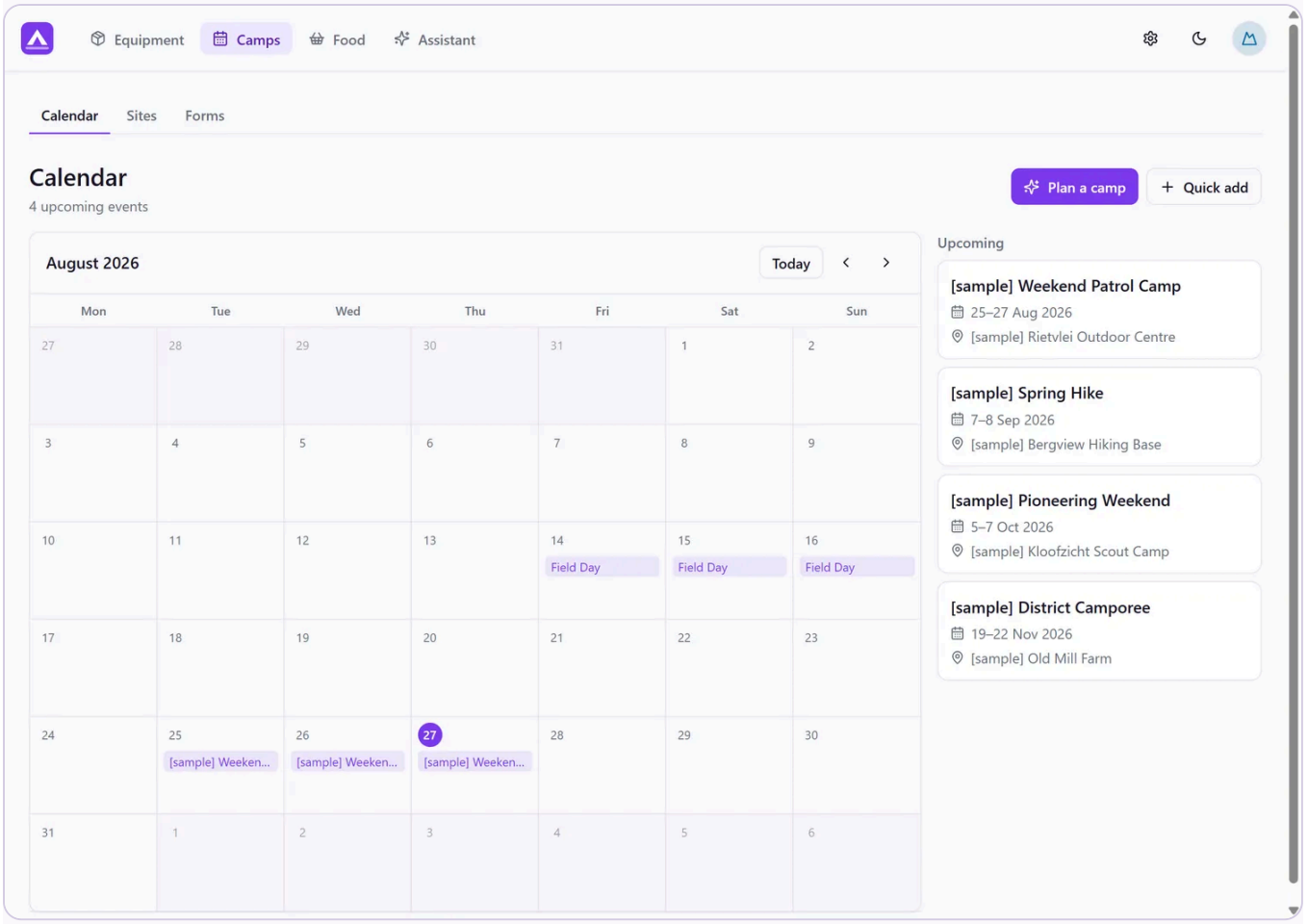
Loans record who took it, from which Scout group, their position and contacts, when it went out and when it's due. A loan with no `returned_on` is still outstanding.



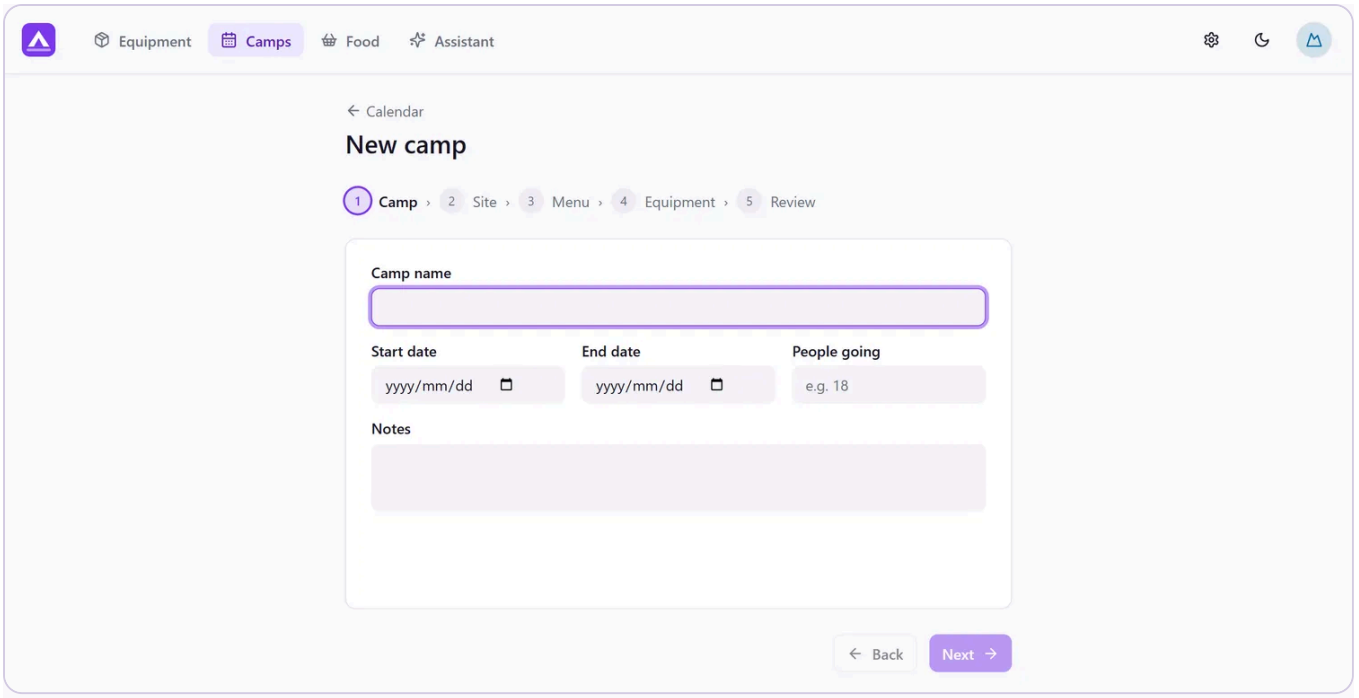
Borrowed equipment. Status is derived, not stored.

Status is worked out in `src/lib/loans.ts` by comparing `yyyy-mm-dd` strings rather than `Date` objects — they sort correctly as text and it sidesteps a timezone shifting a due date by a day. An item is *due soon* within `DUE_SOON_DAYS` (3) of its due date. Quantities on loan are subtracted from the equipment list's availability, clamped at zero so a miscount shows `0` rather than a negative.

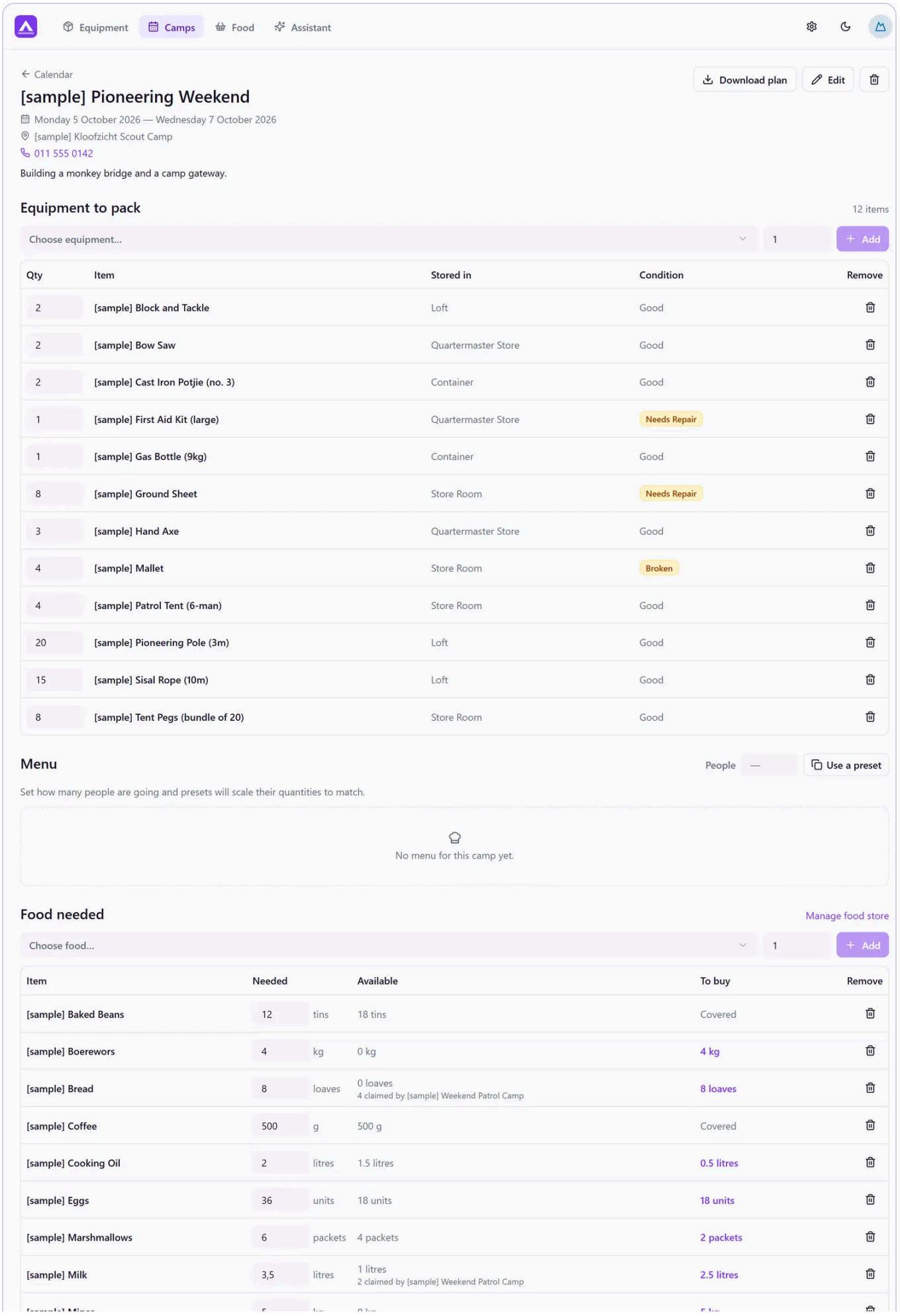
Camps

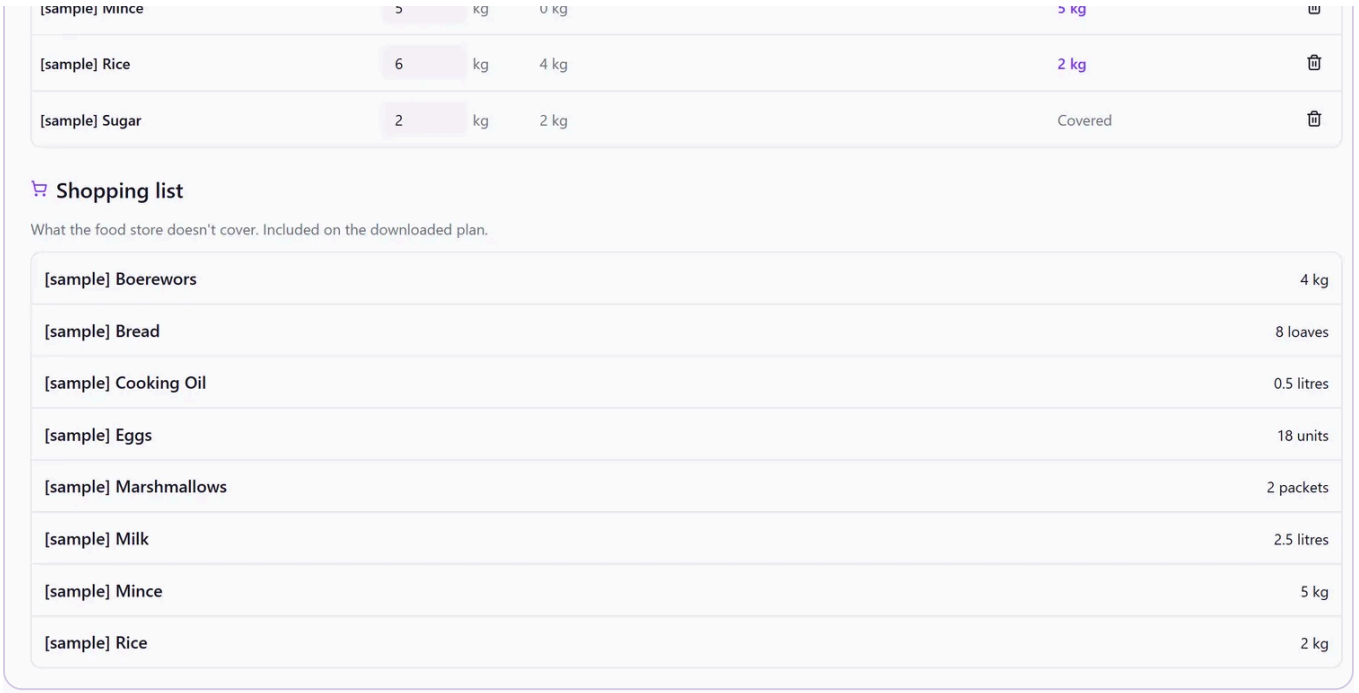


A month grid plus what's coming up. "Plan a camp" opens the wizard; "Quick add" is for troop nights.



The wizard: Camp → Site → Menu → Equipment → Review, creating the event, its packing list and its menu in one pass.


Equipment
Camps
Food
Assistant
Calendar
[sample] Pioneering Weekend
Download plan
Edit
Monday 5 October 2026 — Wednesday 7 October 2026
[sample] Kloofzicht Scout Camp
011 555 0142
Building a monkey bridge and a camp gateway.
Equipment to pack
12 items
Choose equipment...
1
Add
Qty Item Stored in Condition Remove
2 [sample] Block and Tackle Loft Good
2 [sample] Bow Saw Quartermaster Store Good
2 [sample] Cast Iron Potjie (no. 3) Container Good
1 [sample] First Aid Kit (large) Quartermaster Store Needs Repair
1 [sample] Gas Bottle (9kg) Container Good
8 [sample] Ground Sheet Store Room Needs Repair
3 [sample] Hand Axe Quartermaster Store Good
4 [sample] Mallet Store Room Broken
4 [sample] Patrol Tent (6-man) Store Room Good
20 [sample] Pioneering Pole (3m) Loft Good
15 [sample] Sisal Rope (10m) Loft Good
8 [sample] Tent Pegs (bundle of 20) Store Room Good
Menu
People —
Use a preset
Set how many people are going and presets will scale their quantities to match.
No menu for this camp yet.
Food needed
Manage food store
Choose food...
1
Add
Item Needed Available To buy Remove
[sample] Baked Beans 12 tins 18 tins Covered
[sample] Boerewors 4 kg 0 kg 4 kg
[sample] Bread 8 loaves 0 loaves
4 claimed by [sample] Weekend Patrol Camp
8 loaves
[sample] Coffee 500 g 500 g Covered
[sample] Cooking Oil 2 litres 1.5 litres 0.5 litres
[sample] Eggs 36 units 18 units 18 units
[sample] Marshmallows 6 packets 4 packets 2 packets
[sample] Milk 3,5 litres 1 litres
2 claimed by [sample] Weekend Patrol Camp
2.5 litres



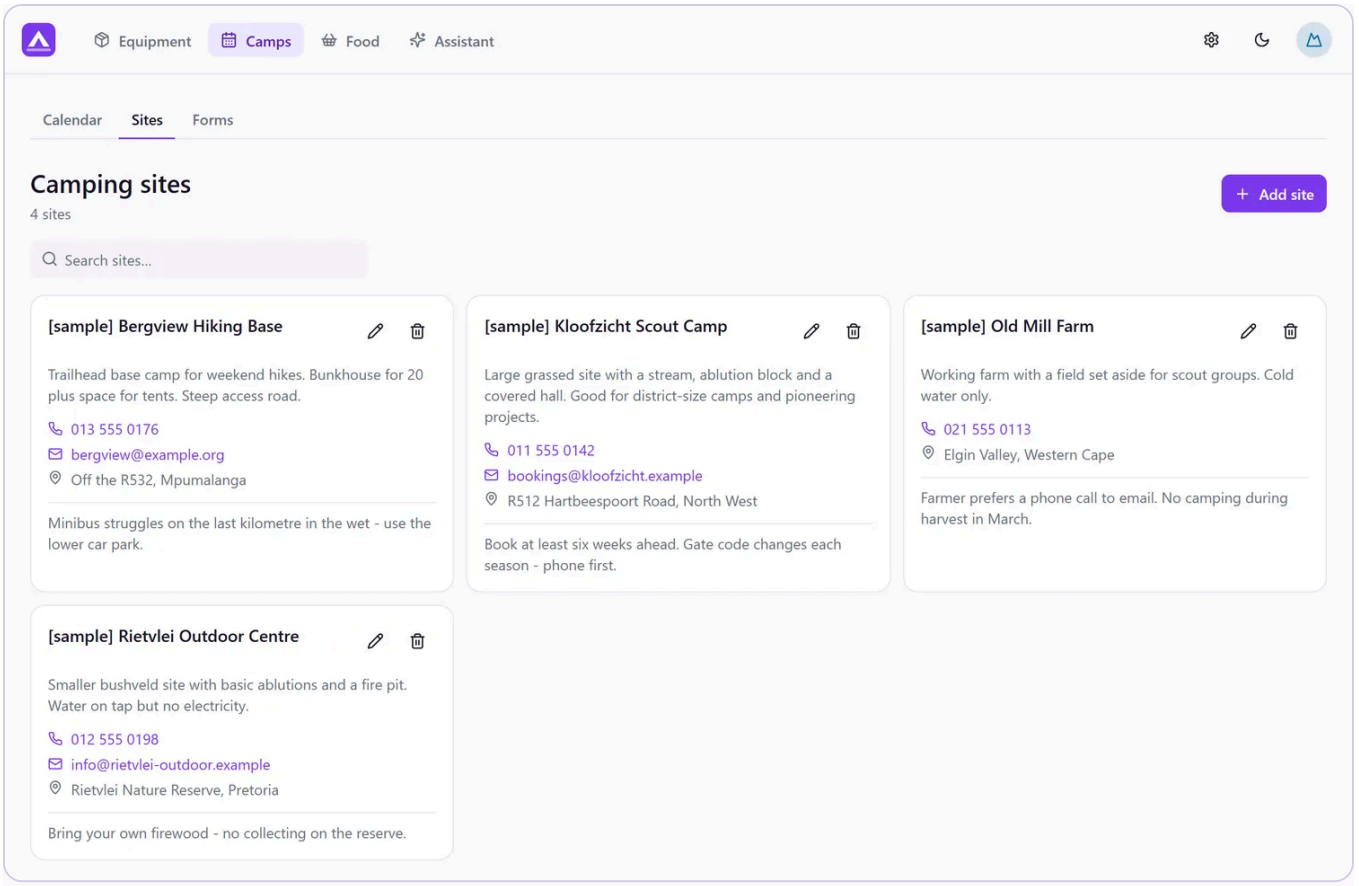
A camp with a packing list, a food plan and a derived shopping list.

Three things in that screen are worth pointing out.

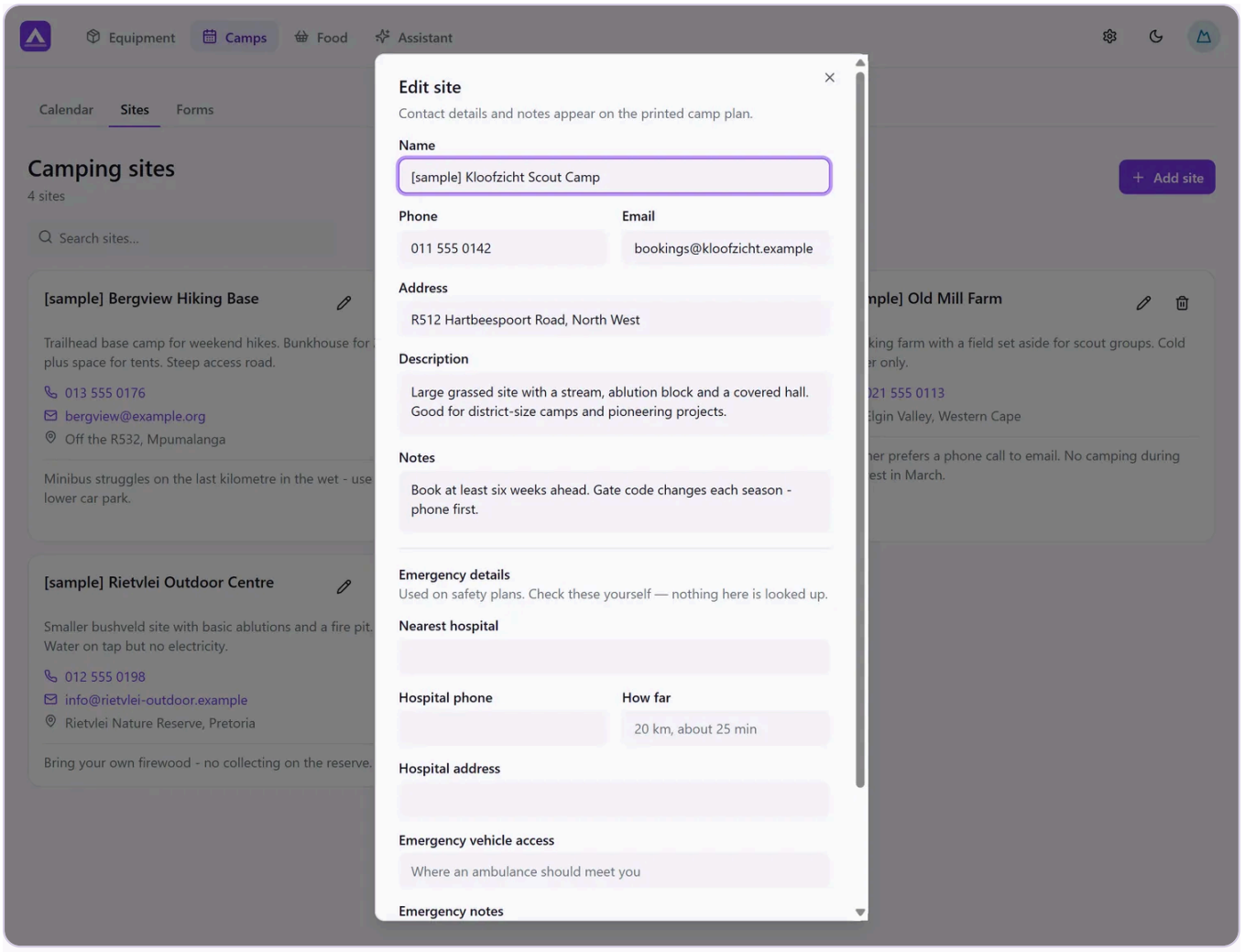
- **Condition travels with the packing list.** A *Needs Repair* or *Broken* item shows its badge here, so it's visible at packing time rather than at camp.
- **Other camps' claims are subtracted.** `[sample] Bread` shows "*4 claimed by [sample] Weekend Patrol Camp*". Without this, two camps a fortnight apart would each see the full cupboard and neither shopping list would mention the bread the other is taking.
- **The shopping list is derived, never typed.** It is the food plan filtered to lines where `toBuy > 0`, where `toBuy = needed - (inStock - claimedElsewhere)`.

Camping sites

Beyond contact details, each site carries what a safety plan needs: nearest hospital, its phone and address, how far it is, emergency vehicle access, and free-text emergency notes.

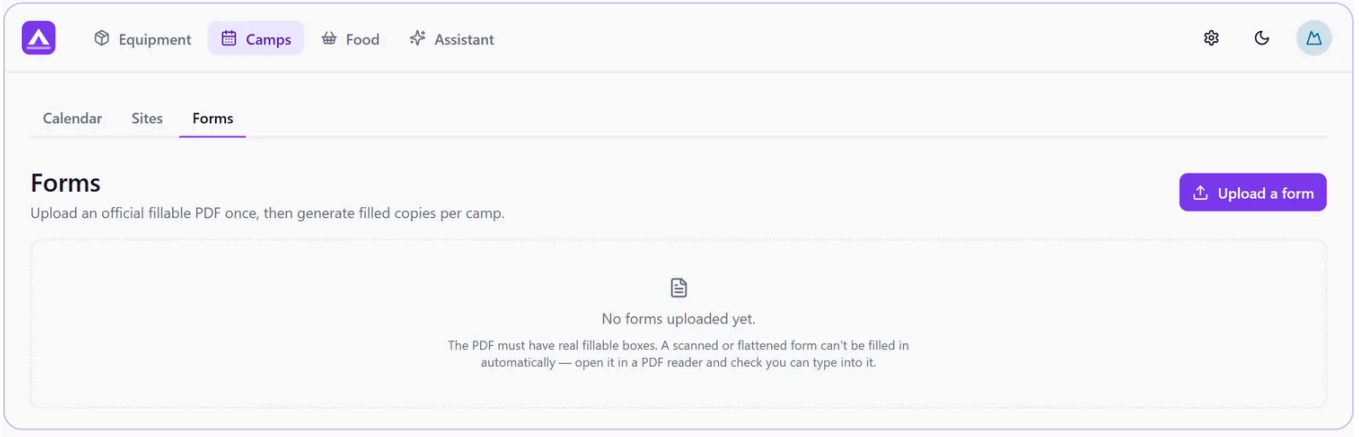


The site directory. The card carries the booking details only.



The emergency half of the site form, where the hospital and access fields live — with the app's own warning that none of it is looked up for you.

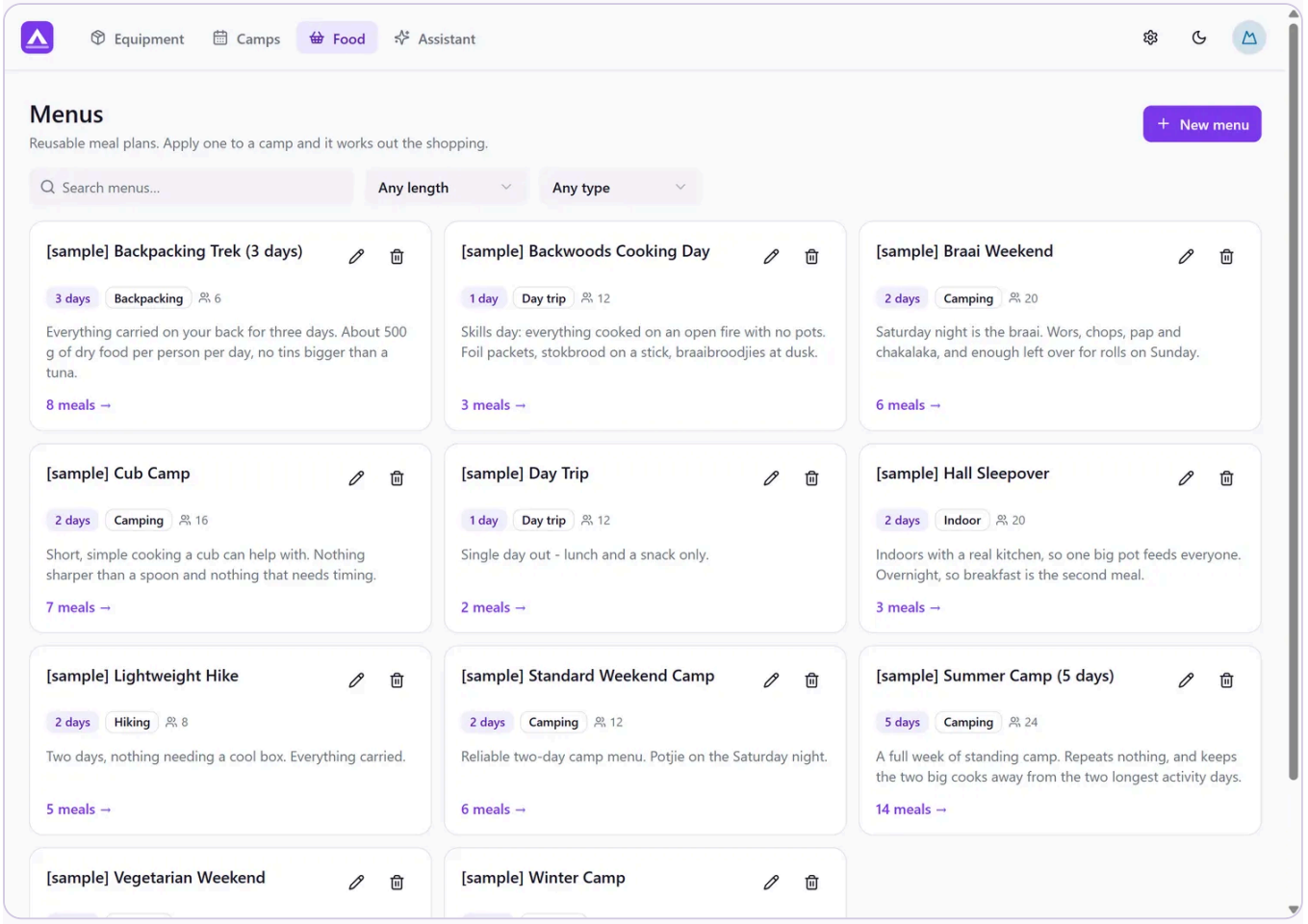
Forms



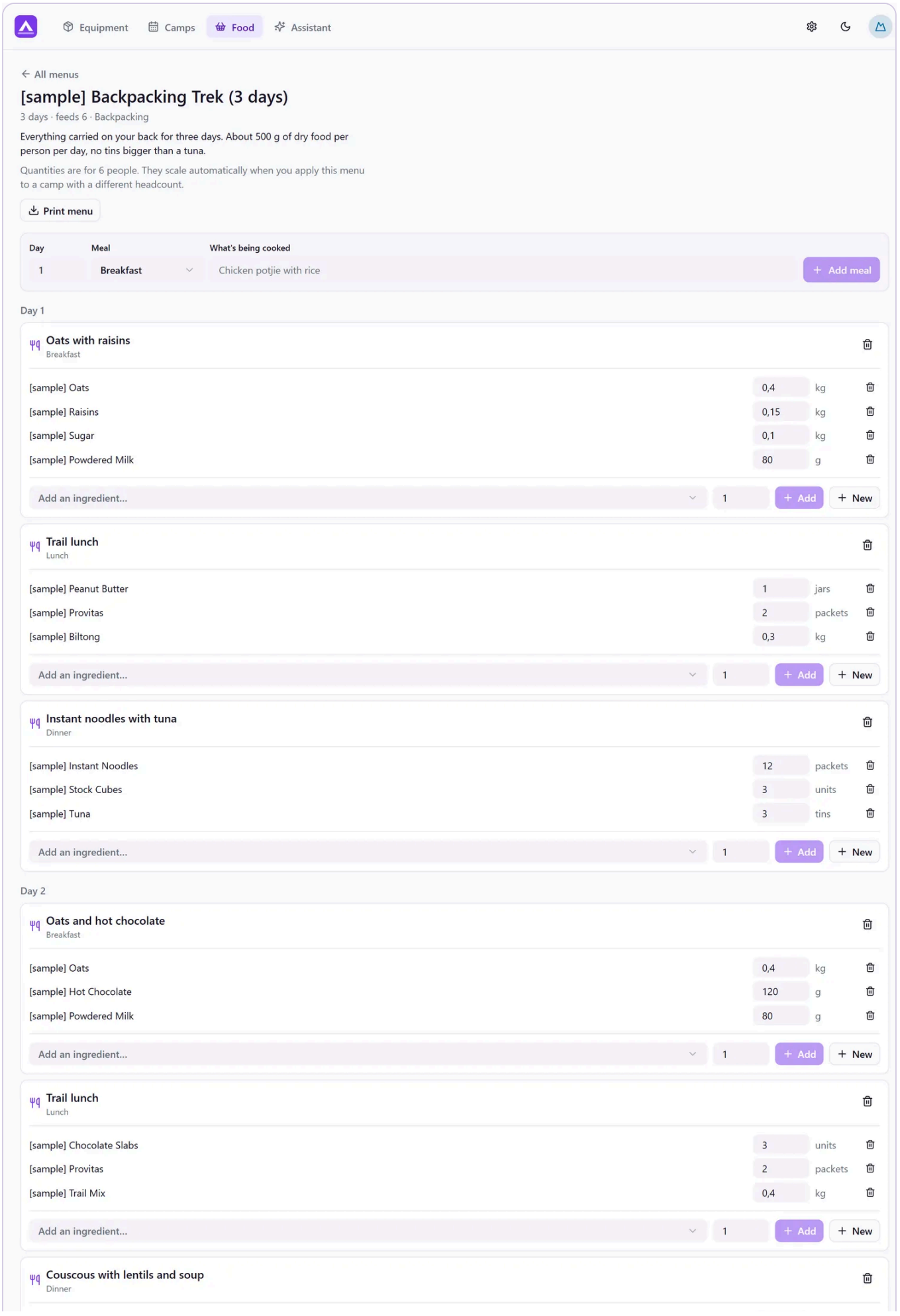
Upload a fillable PDF once, map its fields to camp data, then generate a filled copy per camp. Nothing is uploaded in this project yet, so this is the empty state — which is also where it warns that a scanned or flattened PDF can't be filled in automatically.

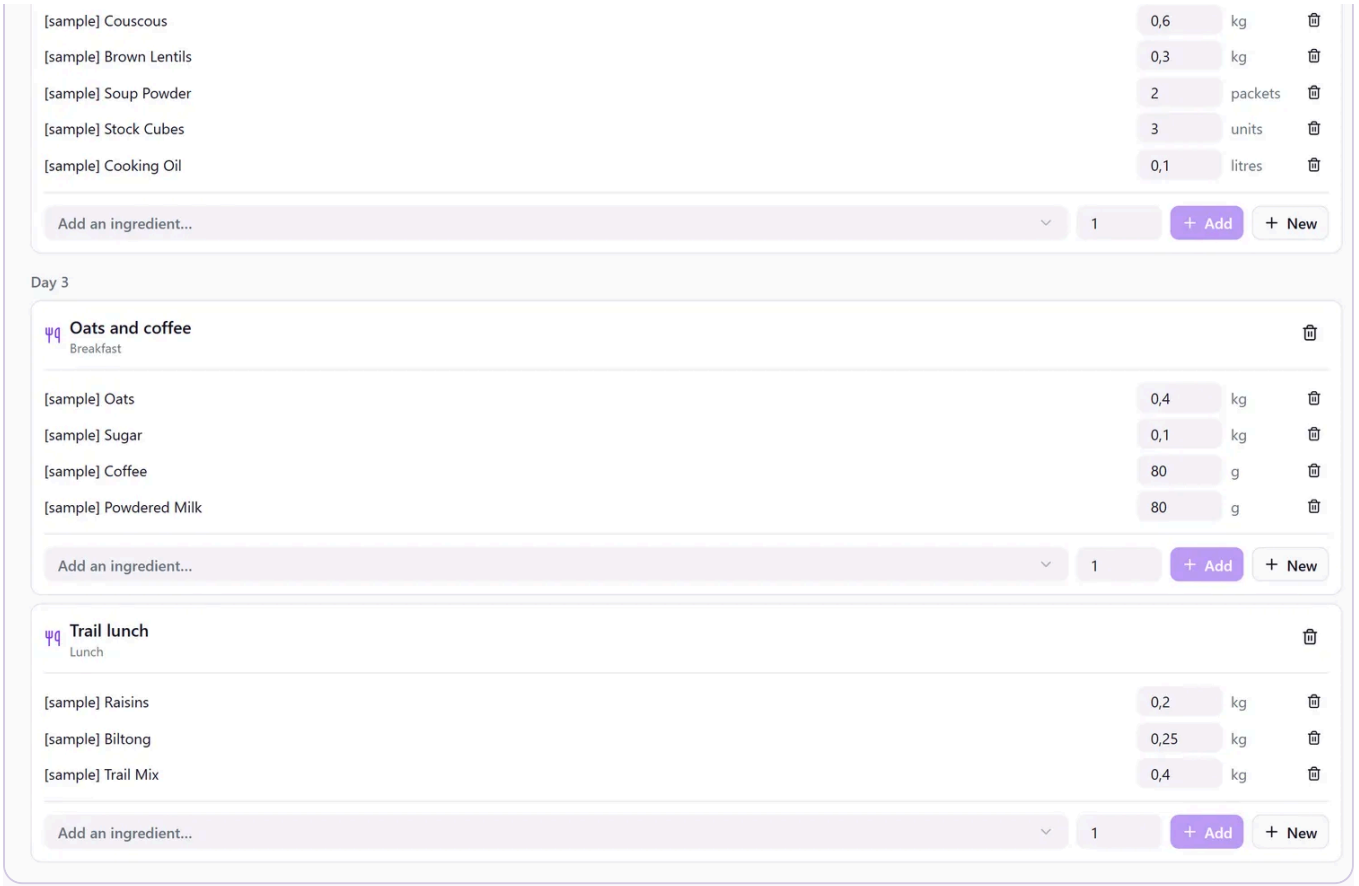
Food and menus

The food store is a plain inventory in whatever units suit — kg, tins, litres, packets, loaves. Menus are meal plans: meals per day, ingredients per meal, written for a stated number of people. A menu with `event_id = null` is a reusable **preset**; one with an `event_id` belongs to a camp. Applying a preset scales every quantity by `headcount / serves`.



The menu library, filterable by length and type. Each card carries its length, its kind, who it serves and how many meals it defines.


Equipment
Camps
Food
Assistant
All menus
[sample] Backpacking Trek (3 days)
3 days · feeds 6 · Backpacking
Everything carried on your back for three days. About 500 g of dry food per person per day, no tins bigger than a tuna.
Quantities are for 6 people. They scale automatically when you apply this menu to a camp with a different headcount.
Print menu
Day
Meal
What's being cooked
1
Breakfast
Chicken potjie with rice
Add meal
Day 1
Oats with raisins
Breakfast
[sample] Oats 0,4 kg
[sample] Raisins 0,15 kg
[sample] Sugar 0,1 kg
[sample] Powdered Milk 80 g
Add an ingredient...
1
Add
New
Trail lunch
Lunch
[sample] Peanut Butter 1 jars
[sample] Provitas 2 packets
[sample] Biltong 0,3 kg
Add an ingredient...
1
Add
New
Instant noodles with tuna
Dinner
[sample] Instant Noodles 12 packets
[sample] Stock Cubes 3 units
[sample] Tuna 3 tins
Add an ingredient...
1
Add
New
Day 2
Oats and hot chocolate
Breakfast
[sample] Oats 0,4 kg
[sample] Hot Chocolate 120 g
[sample] Powdered Milk 80 g
Add an ingredient...
1
Add
New
Trail lunch
Lunch
[sample] Chocolate Slabs 3 units
[sample] Provitas 2 packets
[sample] Trail Mix 0,4 kg
Add an ingredient...
1
Add
New
Couscous with lentils and soup
Dinner



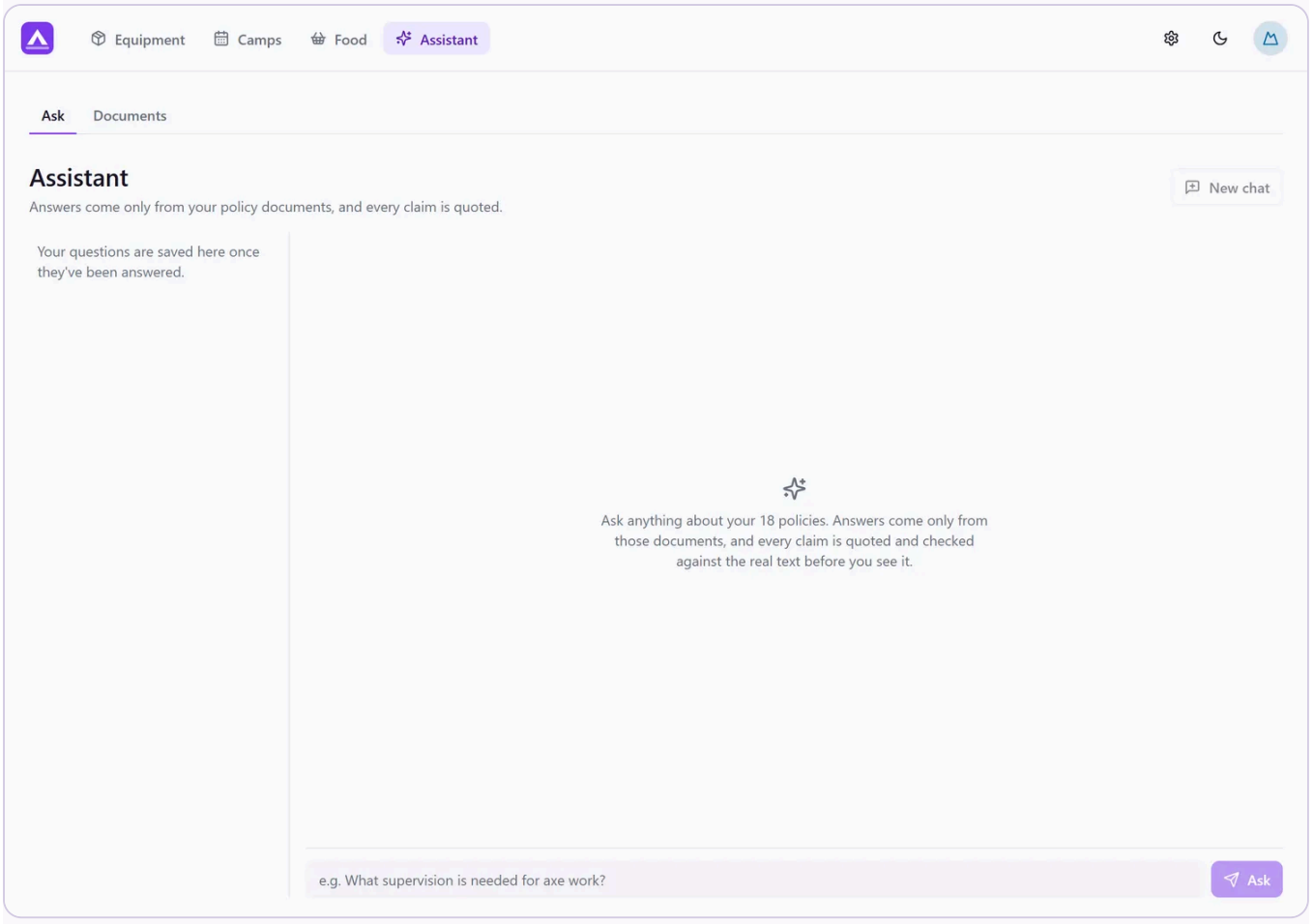
A menu open in the editor: meals grouped by day, ingredients with quantities and units, and the reminder that the quantities are written for six and scale on the way into a camp.

TWO DOCUMENTED RULES, VISIBLE ABOVE

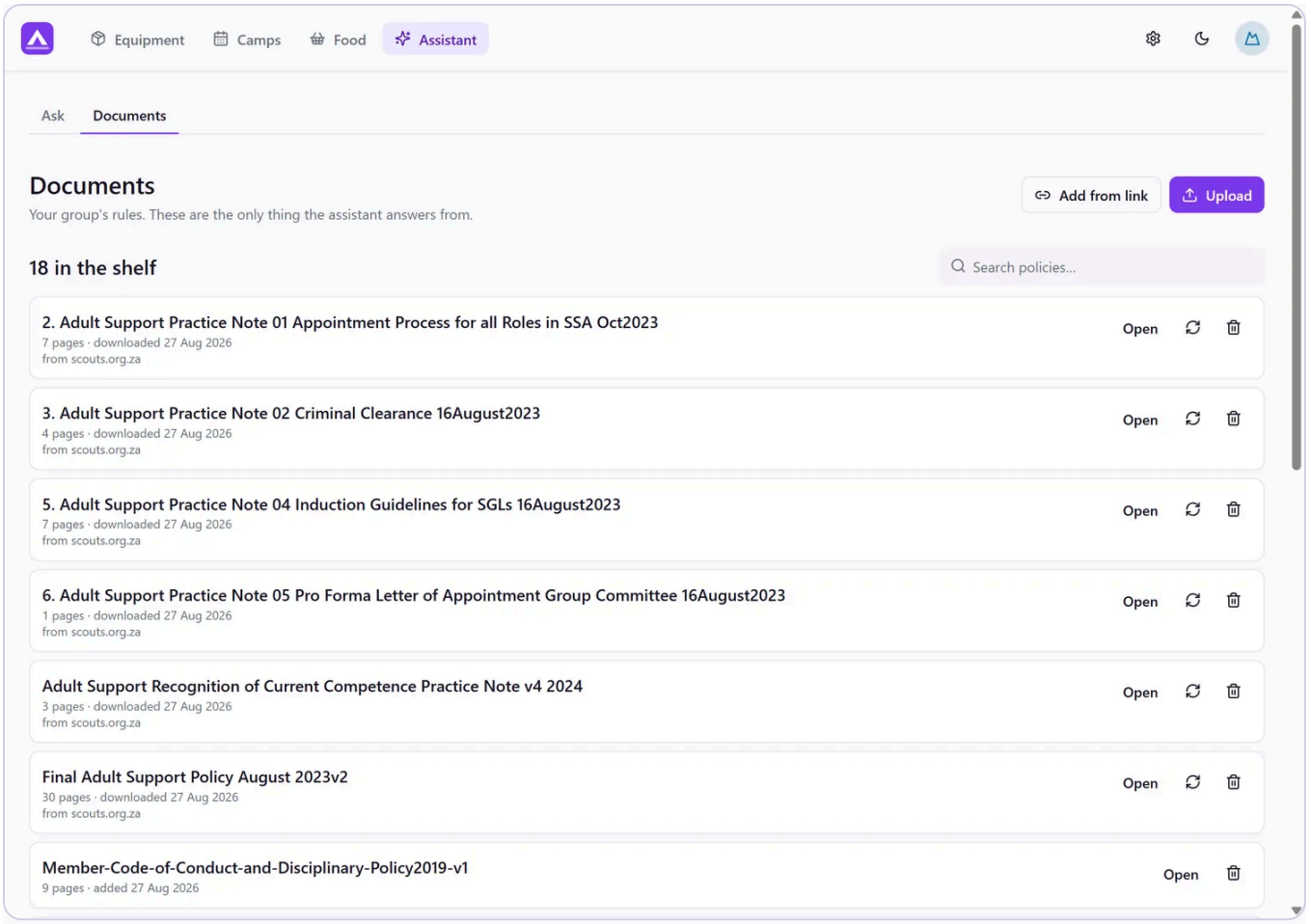
Neither menu screen shows a "Food store / Menus" tab strip, even though `MenusPage` renders one. Food is switched off in this project, which leaves the strip with a single tab — and a lone tab renders nothing. The section itself survives in the nav because Menus, its other tab, is still on.

`aggregateMenuRequirements` sums the same ingredient across every meal that uses it — rice at lunch on day one and dinner on day two is one line on the shopping list, not two — and `mergeRequirements` folds in anything added by hand on the event. Two Postgres functions do the copying server-side so it is atomic: `apply_menu_preset` and `save_menu_as_preset`.

Assistant



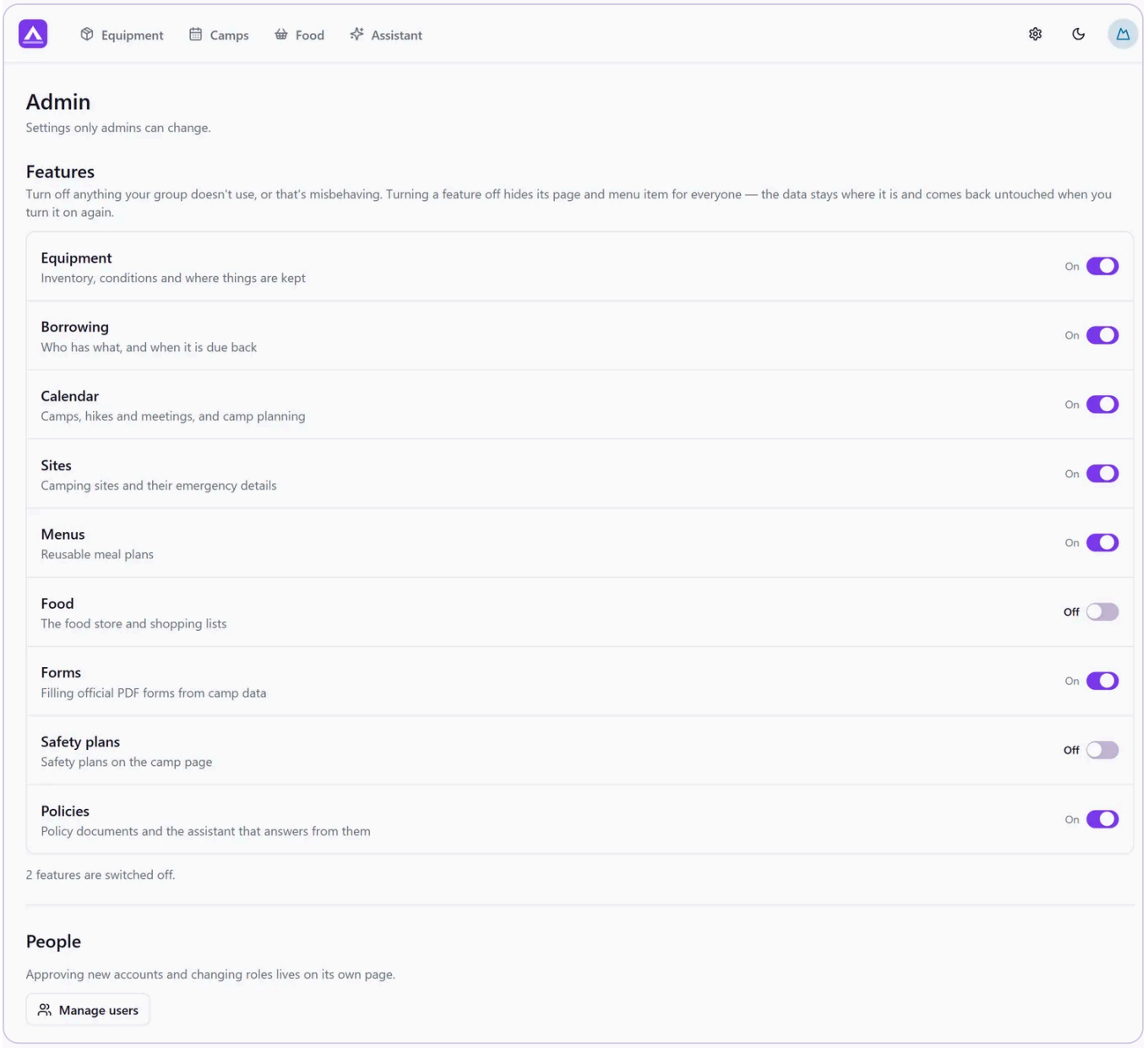
The Ask tab with no conversation started. Saved chats sit in the rail on the left, and the empty state names how many documents are loaded — an answer can only ever come from those.



Eighteen real Scouts South Africa policies. Rows added by link show "downloaded … from scouts.org.za" and carry a ↻ that re-fetches them in place; the uploaded one shows "added" and has no ↻, because there's no source to re-fetch from.

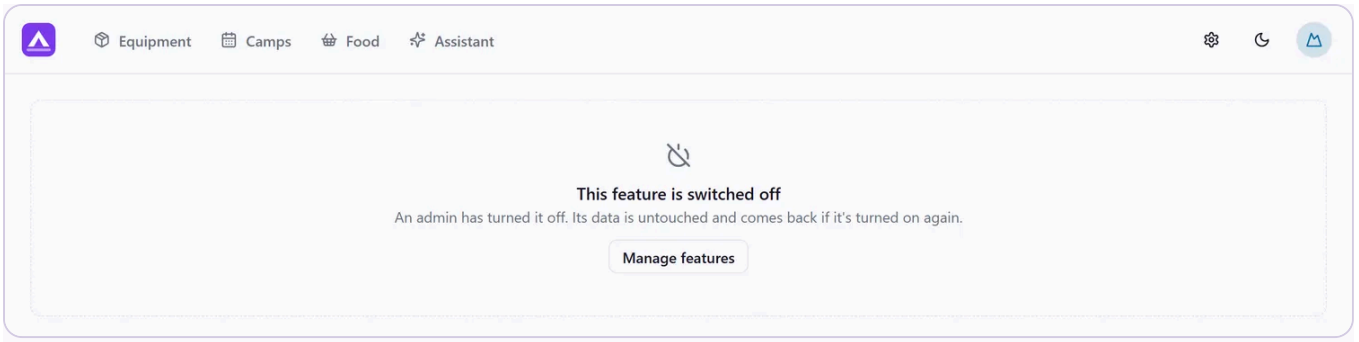
Conversations are saved and **private to whoever asked** — enforced by row-level security on `policy_chats`, not just by the UI. Only the last three exchanges are replayed to the model: enough for a follow-up like *"and what about cubs?"* without pushing the documents down a prompt that grows every turn.

Admin and users

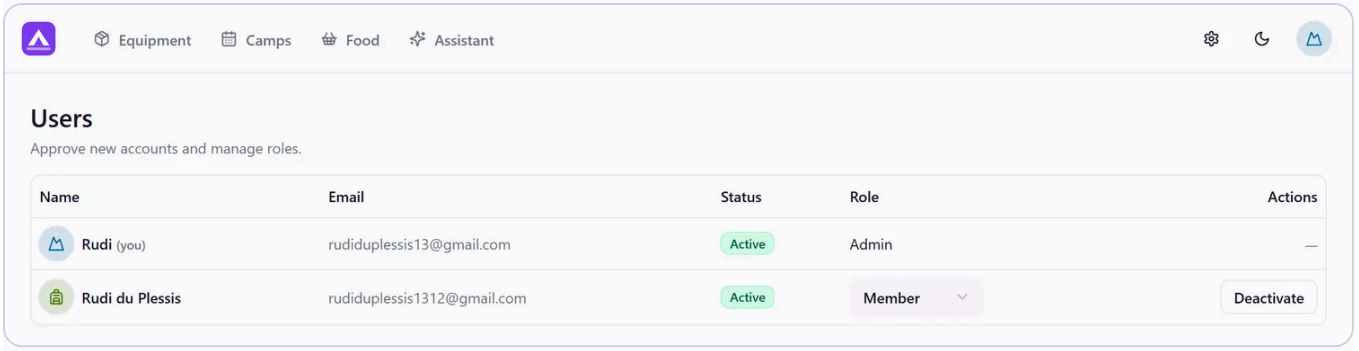


Feature switches. Turning one off asks for confirmation; turning it back on doesn't.

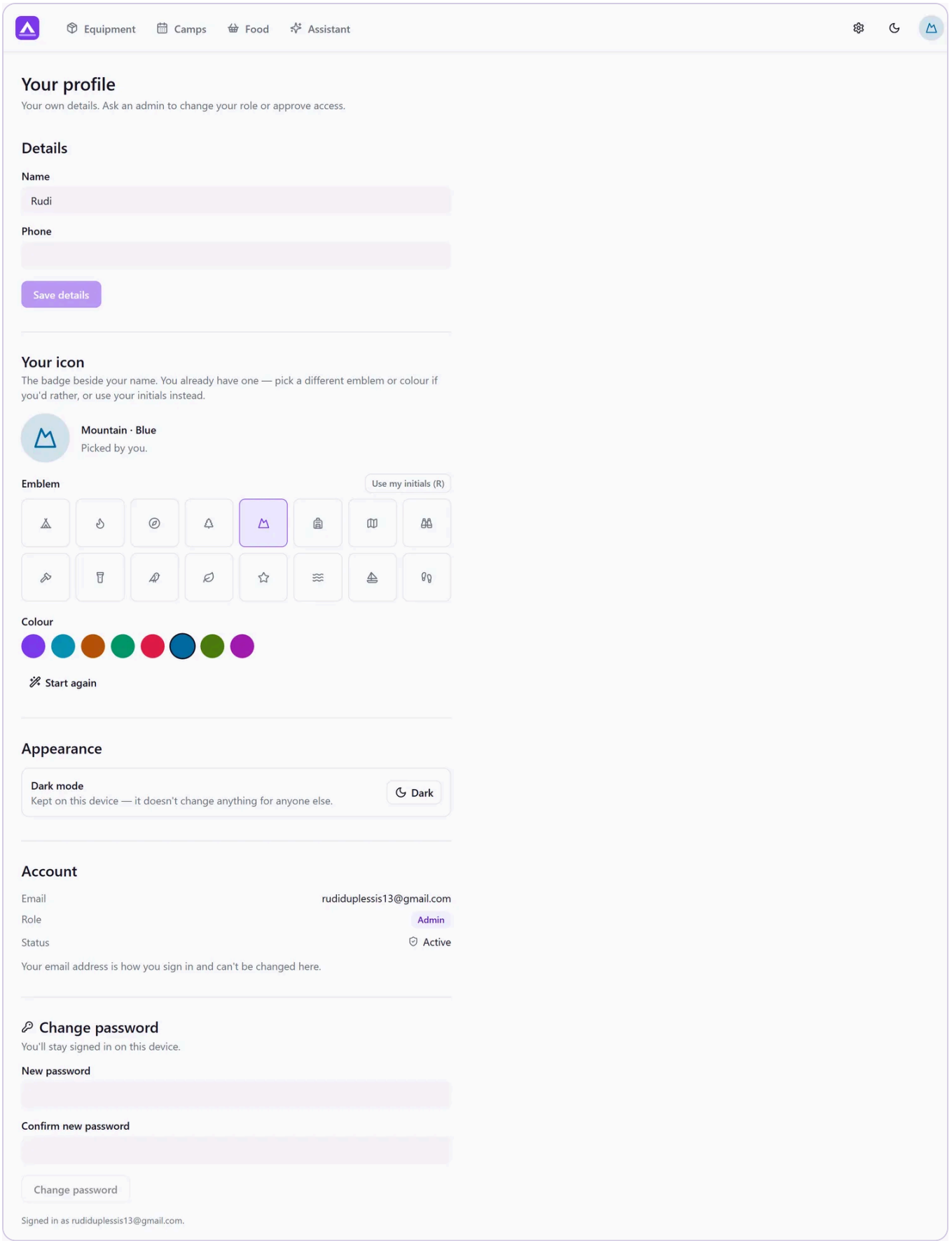
Making both directions equally hard would just train people to click through. The data is untouched either way. A route whose feature is off renders this rather than 404ing:



A switched-off feature, with a way back to the switches for an admin.



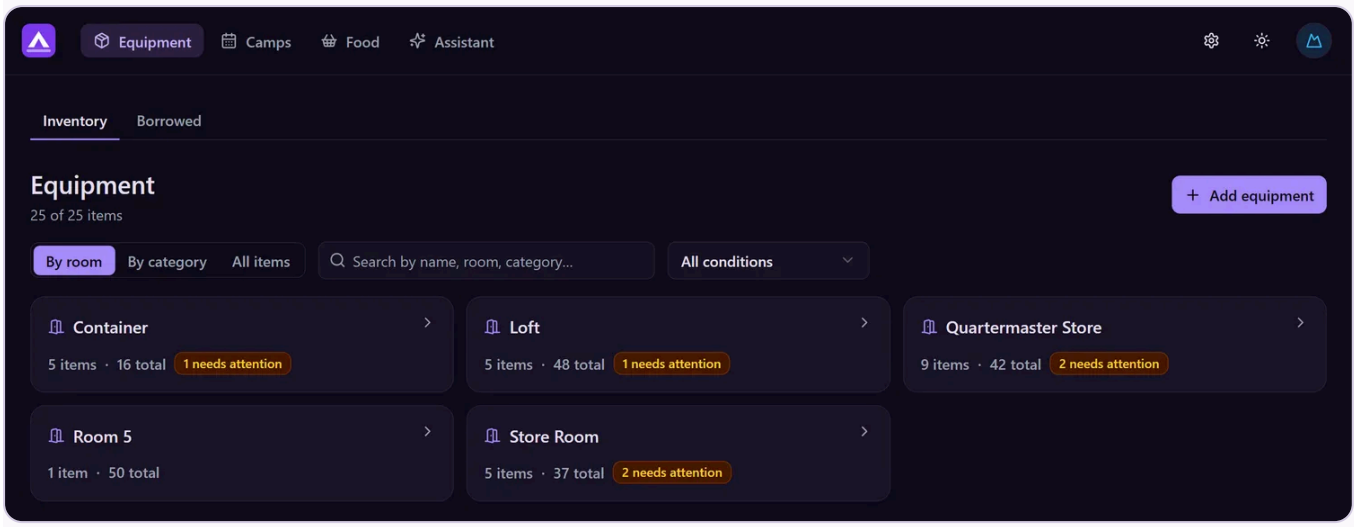
Approving signups, changing roles, deactivating accounts. Admin only at both the route and the database.



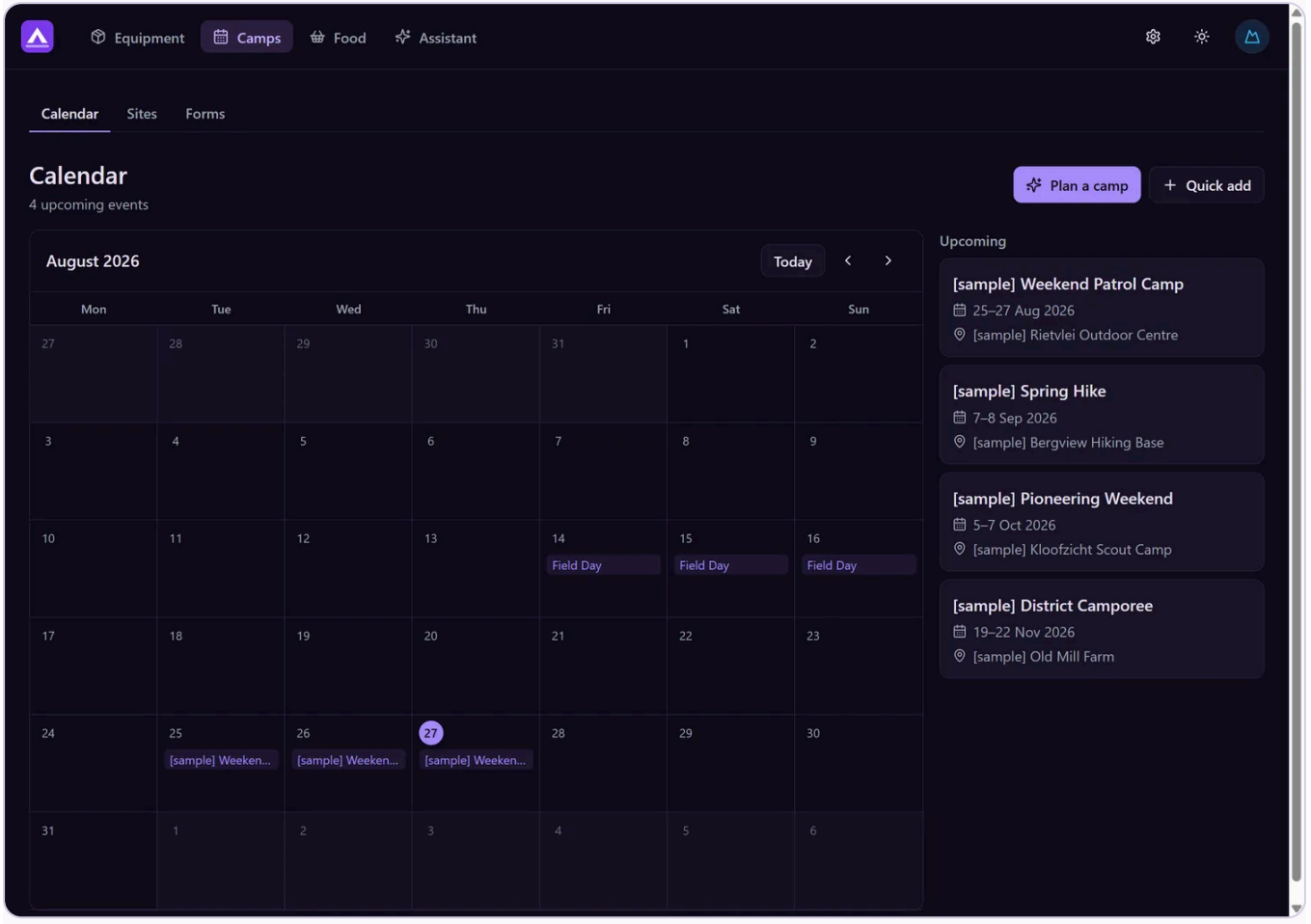
Your profile: details, appearance including the icon picker, and account settings.

Dark mode and mobile

The theme is a class on `<html>`, persisted to `localStorage`. Every colour is a CSS variable, including the eight person-icon hues, so they swap together.

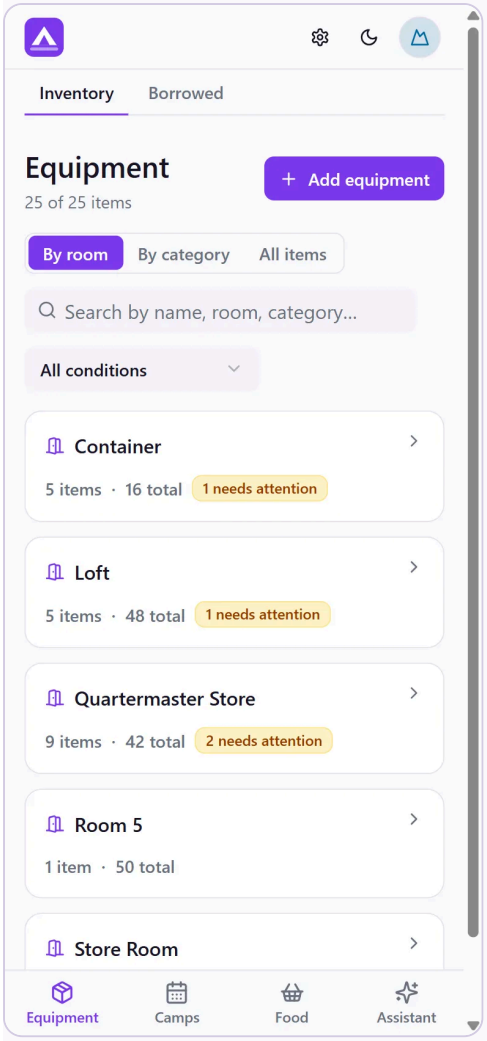


Equipment, dark.

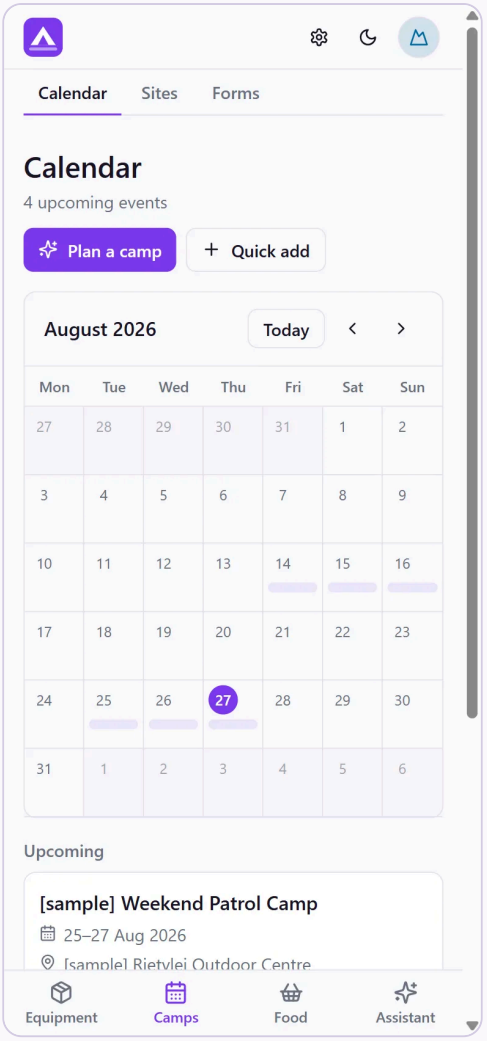


The calendar, dark.

On a phone the header nav is hidden and a fixed bottom bar takes over — four wide targets that are easier to reach one-handed than a row that scrolls off the edge.



Equipment on a phone.



The calendar on a phone.

03 Roles and permissions

Four roles and three account statuses.

```
status:  Pending  →  Active     (an Admin approves)
                  →  Rejected   (an Admin rejects, or later deactivates)

role:    Viewer   (the default every new account gets)
         Member
         Manager
         Admin
```

Only an **Active** account can do anything at all. Pending and Rejected accounts fail `is_active_user()` in every policy, so they are blocked at the database, not merely redirected.

CAPABILITY	ADMIN	MANAGER	MEMBER	VIEWER
Read everything	✓	✓	✓	✓
Add / edit equipment, camps, sites, food, menus, loans, forms	✓	✓	✓	—
Delete those	✓	✓	—	—
Add / edit / delete policy documents	✓	✓	—	—
Ask the assistant	✓	✓	✓	✓
Approve signups, change roles, deactivate	✓	—	—	—
Toggle feature switches	✓	—	—	—

> **THE ONE ASYMMETRY THAT MATTERS**
>
> Only Admins and Managers can change **policy documents**, even though Members can edit everything else. An answer is only as trustworthy as who can edit what it answers from.

Own-row exceptions. Anyone can read and update their own `users` row — name, phone, icon choice — but a trigger reverts any attempt to change their own `role` or `status`. Chats are stricter still: a chat and its messages are visible only to the person who owns them, whatever their role.

04 Architecture

Stack

LAYER	CHOICE
UI	React 18.3, TypeScript 5.4
Build	Vite 6.4, `vite-plugin-pwa` 0.21
Styling	Tailwind CSS v4, CSS variables for theming
Components	Radix UI primitives in the shadcn/ui pattern, `lucide-react`
Routing	React Router 7.18, `createBrowserRouter`
Backend	Supabase — Postgres, Auth, RLS, Realtime, Storage, Edge Functions
PDFs	`jspdf` to write, `pdf-lib` to fill, `pdfjs-dist` to read — all lazy
Dates	`date-fns` 3.6

There is no state-management library, no data-fetching library and no test framework. State lives in five context providers and one reusable realtime hook.

Directory map

```
BaseMind3.0-main/
├── README.md                   project background and history
├── DOCUMENTATION.md            the long-form version of this page
├── docs/screenshots/           the images used here
└── basemind-app/
    ├── index.html              the shell; PWA manifest link injected at build
    ├── vite.config.ts          aliases, PWA/workbox config, chunk splitting
    ├── vercel.json             build command, output dir, SPA rewrite
    ├── .env.example            the two variables you must set
    ├── supabase/
    │   ├── migrations/         0001 … 0011, run in order, once each
    │   ├── seed/               dummy_data.sql, remove_dummy_data.sql
    │   └── functions/          three Deno Edge Functions, deployed by hand
    └── src/
        ├── App.tsx             provider stack + missing-config screen
        ├── routes.tsx          router, lazy imports, feature/role gates
        ├── types/database.ts   every row type, and the typed Database generic
        ├── context/            Auth, Equipment, Features, Offline, Theme
        ├── lib/                all business logic, framework-free
        └── components/         ui/ primitives, layout/, one folder per feature
```

The provider stack

```
ThemeProvider              light/dark class on <html>, persisted
 └── (missing env vars? → render the config screen instead of the app)
     └── OfflineProvider   online/offline, the outbox, sync on reconnect
         └── AuthProvider  session, profile, role checks, cached fallback
             └── FeaturesProvider  the feature_flags table, live
                 └── RouterProvider + Toaster
```

The ordering is load-bearing. `AuthProvider` sits inside `OfflineProvider` because the cached profile is what keeps an already-signed-in leader from being bounced to the login screen with no signal. `FeaturesProvider` sits inside `AuthProvider` because reading `feature_flags` requires an active session.

If either environment variable is missing, the whole router is replaced by a screen naming what's absent. The Supabase client falls back to a dummy URL, because

`createClient` throws on an empty one and that would blank the page before React could explain anything.

Routing

The three auth pages are eagerly loaded — they're the first thing anyone sees, so splitting them would only add a spinner to the critical path. **Everything behind the login is lazy**, which keeps the initial download small on a phone with camp-grade signal.

`AppLayout` is the real access gate: no session → `/login`; no profile or `status ≠ "Active"` → `/pending`.

> **WORTH KNOWING**
>
> `/equipment` has **no** feature gate on the route, even though an `equipment` flag exists. Turning that flag off removes the Inventory tab and the section link, but the route still renders — and `/` still redirects to it.

Navigation structure

Four sections in the bar, not eight destinations — eight items never fitted a phone. Related pages are tabs *within* a section.

SECTION	TABS
Equipment	Inventory `/equipment`, Borrowed `/loans`
Camps	Calendar `/events`, Sites `/sites`, Forms `/forms`
Food	Food store `/food`, Menus `/menus`
Assistant	Ask `/assistant`, Documents `/policies`

A section's tabs are its source of truth: it survives as long as **any one** of them is switched on, and points at the first one that is. Gating a section on a single feature meant turning off Food also hid Menus, which sits in the same section but has its

own switch. A strip with one remaining tab renders nothing. Admin is deliberately not in this bar — settings live in the header, never pushed into an overflow menu.

The realtime data pattern

Almost every page uses one hook:

```
const { rows, loading, error } = useRealtimeTable("events", { orderBy: "start_date"
```

1. Reads the last known rows from IndexedDB and renders them immediately. At camp this is often the only data there is, and even on a good connection it beats a spinner.
2. Fetches from Supabase, sorts, renders, writes the fresh copy back to IndexedDB.
3. Subscribes to `postgres_changes` and folds INSERT/UPDATE/DELETE into local state.
4. Suppresses the fetch error entirely if there was a usable cache — the offline banner already explains the situation.

> THE CONSEQUENCE TO INTERNALISE
>
> **Mutations do not update local state.** They write to Supabase per row and let the realtime event come back and do it. That's what keeps two leaders' tabs in step, and it's what the old app got wrong with whole-table rewrites.

Recovering from a deploy that lands mid-session

A real failure mode for a long-lived tab: the app is code-split into chunks whose filenames contain a build hash, so after a deploy an open page still refers to the *previous* build's filenames — and those are gone. The next navigation throws `Failed to fetch dynamically imported module` and the app falls over.

- `installStaleBuildHandler()` listens for `unhandledrejection` and `error`, catching failures outside React's error boundary — a hover-triggered prefetch, or a lazy import that rejects between renders.

- `RouteError` catches the same thing inside the router.
- Recovery is a single reload, recorded in `sessionStorage`, so a genuinely broken build shows a real error instead of reloading forever.
- The service worker uses `skipWaiting`, `clientsClaim` and `cleanupOutdatedCaches`, so the new build takes over immediately.

---

05 The data model

Eighteen tables. Every one has RLS enabled. Fifteen are in the `supabase_realtime` publication — the exceptions are `users`, `policy_chats` and `policy_chat_messages`, none of which benefit from being broadcast.

```
auth.users ──(trigger)──▶ public.users
                              │
                              ├── equipment.checked_by
                              ├── policies.created_by
                              └── policy_chats.user_id ──▶ policy_chat_messages.chat_

equipment ──▶ equipment_loans          (who has what, and when it's due)
          └──▶ event_equipment ◀── events            (the packing list)

camping_sites ◀── events ──▶ event_food ──▶ food_items   (the food plan)
      │              │
      │              ├──▶ menus ──▶ menu_meals ──▶ menu_meal_items ──▶ food_items
      │              └──▶ safety_plans ──▶ safety_scenarios
      └── hospital_*, emergency_* columns feed the safety plan

camp_forms          (an uploaded fillable PDF + its field mapping)
feature_flags       (the switches on the admin page)
policies            (documents + extracted text)
```

TABLE	MIGRATION	WHAT IT HOLDS
`users`	0001	Profile mirror of `auth.users`: name, phone, role, status, icon choice
`equipment`	0001	Name, category, quantity, location, condition, notes, check dates
`camping_sites`	0003, 0006	Venue contacts; hospital and emergency fields added in 0006
`events`	0003, 0005	Camps, hikes, meetings; `headcount` added in 0005
`event_equipment`	0003	The packing list — event ↔ equipment with a quantity
`food_items`	0003	The food store: name, unit, quantity, category
`event_food`	0003	Food required for an event
`equipment_loans`	0004	Borrower, group, contacts, dates; `returned_on` null = still out
`menus`	0005	`event_id = null` → reusable preset; otherwise a camp's own menu
`menu_meals`	0005	Day number, meal slot, name, notes
`menu_meal_items`	0005	Ingredient lines: meal ↔ food item + quantity
`safety_plans`	0006	Leader, standby, first aider, emergency numbers, generated text
`safety_scenarios`	0006	Per-situation response, destination, contact, sort order
`camp_forms`	0007	Uploaded PDF's storage path + `field_map`
`feature_flags`	0008, 0009	Nine switches, keyed by `key` rather than `id`
`policies`	0009, 0011	Title, category, storage path, extracted `content`, source URL
`policy_chats`	0011	One saved conversation, owned by a user

TABLE	MIGRATION	WHAT IT HOLDS
`policy_chat_messages`	0011	Turns within a chat; assistant rows carry the verified result

Conventions

- **Primary keys** are `uuid`, except `users.id` (references `auth.users`) and `feature_flags.key` (text).
- `updated_at` is maintained by a shared trigger, not by the client.
- **Calendar dates** are `date`, compared as `yyyy-mm-dd` strings in the app to avoid timezone drift.
- **Quantities** go through `roundQuantity()` wherever they're summed, to shake off artefacts like `0.30000000000000004`.

> **A TRAP IN THE TYPE DEFINITIONS**
>
> Row types must be `type` aliases, never `interface`. Supabase's client generic constrains every row to `Record<string, unknown>`, and an interface has no implicit index signature — using one silently degrades every insert/update argument to `never`.

06 The security model

This is the part the rebuild exists for, so it's worth reading properly.

Signup cannot elevate itself

`handle_new_user()` is an `after insert` trigger on `auth.users`. It creates the profile row with role and status **hard-coded** to `'Viewer'` and `'Pending'`, ignoring anything the client put in its signup metadata. There is deliberately **no INSERT**

policy on `public.users` — the trigger is `security definer` and bypasses RLS; client inserts are default-denied.

Two helper functions decide everything

```
public.current_role()    -- the caller's role, or NULL
public.is_active_user()  -- true only when the caller's status is 'Active'
```

Both are `security definer` and `stable`. That matters: they must bypass RLS internally, otherwise querying `users` from inside a policy defined on `users` recurses.

The standard policy shape

```
select  →  is_active_user()
insert  →  is_active_user() and current_role() in ('Admin','Manager','Member')
update  →  is_active_user() and current_role() in ('Admin','Manager','Member')
delete  →  is_active_user() and current_role() in ('Admin','Manager')
```

Deviations from that shape, all intentional:

TABLE	DEVIATION
`users`	select: own row *or* any active user. update: own row *or* an Admin. delete: Admin only. No insert policy.
`policies`	insert/update/delete require **Admin or Manager** — Members cannot change what the assistant answers from.
`feature_flags`	select: any active user (the nav needs them). update: Admin only. No insert or delete — the set of features is defined by migrations.
`policy_chats`, `policy_chat_messages`	Scoped to `user_id = auth.uid()` for every operation, regardless of role. Messages check ownership through their parent chat.

Column-level protection for role and status

RLS's `WITH CHECK` can gate a whole row but not individual columns. A user allowed to update their own row could therefore set `role = 'Admin'` themselves. A trigger closes that:

```
if auth.uid() is not null and public.current_role() is distinct from 'Admin' then
  new.role   := old.role;
  new.status := old.status;
end if;
```

The `auth.uid() is not null` check is the subtle part, and is the entire subject of migration 0002. Statements from the SQL Editor carry no JWT, so `auth.uid()` is NULL and `current_role()` returns NULL — "distinct from Admin" — and without this guard the documented first-admin bootstrap would report success while silently changing nothing.

Storage

BUCKET	CREATED IN	READ	WRITE	DELETE
`camp-forms`	0007	any active user	Admin / Manager / Member	Admin / Manager
`policies`	0009	any active user	Admin / Manager	Admin / Manager

Migration 0010 removes an older `avatars` bucket entirely — person icons are drawn by the app, so nothing needs uploading.

> **THE CHECK THAT ACTUALLY MATTERS**
>
> Sign in as a Viewer, take that session's token, and `POST` directly to the Supabase REST `equipment` endpoint, bypassing the UI entirely. Postgres rejects it. That is the server-side enforcement the original app never had.

07 Offline behaviour

Three separate mechanisms, each covering a different failure.

1 · The app shell

Every JS, CSS, HTML, SVG and WOFF2 asset is precached — the size cap is raised to 5 MB because `pdf-lib` alone is ~440 kB and the default 2 MB cap would skip it. Supabase requests are explicitly `NetworkOnly`: data is cached by the app in IndexedDB, not by the service worker, and serving a stale auth round-trip would be actively harmful.

2 · Reading

`useRealtimeTable` caches each table in IndexedDB and renders from it first. `AuthProvider` caches the profile and falls back to it when the server can't be reached — so an already-signed-in leader with no signal stays signed in. The cached role decides what the UI *offers*, never what the database *allows*.

3 · Writing

An outbox in IndexedDB. `addRow()` inserts directly when online; when offline — or when the error looks like a network failure rather than a rejection — it queues. On reconnect it waits a second for the connection to settle, then replays. Genuine rejections (RLS, constraint violations) surface immediately rather than being queued to fail again later; permanently failed rows are surfaced so they aren't lost silently.

```
canAdd    = Admin | Manager | Member          // allowed offline: queued and replaye
canEdit   = (Admin | Manager | Member) && online
canDelete = (Admin | Manager) && online
```

The split is deliberate. Adding queues safely — two people adding different rows never collide. Editing or deleting the same row *does* collide, and resolving that without asking would mean quietly discarding someone's work. So those are paused until there's a connection to check against, and `blockedByOffline` tells the UI to explain that rather than just hiding the button.

> **FAILING OPEN**
>
> `isEnabled()` returns `true` for any flag it doesn't have. If the flags table can't be read — offline before it was ever cached, or migration 0008 not yet run — hiding the whole app would be far worse than showing a section someone turned off.

08 The AI features, and how they're constrained

Two optional features use Gemini. Both work on the same principle: **the model's instruction not to invent things is not trusted; its output is verified against the source data and discarded if it doesn't match.** Both have a fully working non-AI path.

The API key never reaches the browser. It lives as a Supabase Edge Function secret; a `VITE_` variable would be baked into the bundle and readable by anyone who opens devtools.

FUNCTION	SECRETS	WITHOUT IT
`safety-plan`	`GEMINI_API_KEY`	The "Rewrite more readably" button errors; the template still works
`policy-chat`	`GEMINI_API_KEY`	Documents still upload, search and open; only the question box errors
`fetch-policy-pdf`	none	Uploading files still works; only "add by link" is unavailable

Both AI functions default to `gemini-3.5-flash-lite` — the cheapest class with the most generous free quota, and its thinking level already defaults to minimal so none of the output budget goes on reasoning. Override with a `GEMINI_MODEL` secret. If the name isn't available on your key the call fails with a 404, and the error names both the model it tried **and lists the models your key can actually use**.

> **DEPLOYED BY HAND**
>
> These functions do not update from a git push. After editing one in the repo you must paste it into the Supabase dashboard again.

Safety plans

`collectFacts()` flattens everything a leader typed, plus the site's emergency fields, into labelled pairs. That single list is both what the deterministic template renders *and* what any generated prose is checked against — so the two cannot drift apart.

The optional rewrite is constrained three ways:

1. The key is server-side.
2. The model is told to use only the supplied facts.
3. **Every number in the reply is checked against the facts before it's shown.** A reply containing an unrecognised number is discarded and you keep the plain version.

`safety_plans.generated_by` records `'template'` or `'gemini'` so you can always tell which you're looking at. Whatever produced it, read the plan before you rely on it.

The assistant

Getting text in. Text is extracted from a PDF with `pdf.js` (lazy-loaded — ~350 kB, only needed when someone adds a document). Incidental PDF spacing is collapsed, because citations are later matched against this stored text. A PDF yielding no text

— a scan or a photograph — is **rejected at upload** with an explanation, rather than silently becoming a document that never appears in any answer.

Replacing, not duplicating. A document whose title you already have replaces the existing one; a unique index on `lower(title)` enforces it. Two copies means the assistant quotes whichever it read first, and an edit to one silently doesn't apply.

Adding by link. `fetch-policy-pdf` downloads server-side, because almost nothing serves a PDF with headers that allow a cross-origin read — and because at camp the user's connection is the one you least want to spend on a 4 MB file. It's fetching a URL somebody typed, from inside your infrastructure, which is the classic shape of a server-side request forgery, so it:

- accepts only `http` / `https`
- requires the caller to be an **Active Admin or Manager**, checked against the database
- follows redirects by hand, re-checking the destination at every hop (max 3)
- refuses loopback, private, link-local and cloud-metadata addresses
- requires the response to actually be a PDF, capped at 15 MB

The file's own comments name the residual risk honestly: a public hostname that resolves to a private address (DNS rebinding) isn't caught, because name resolution isn't available there to check.

VERIFYING ANSWERS

This is the core of it. The model returns an answer plus citations, each naming a policy and quoting it verbatim. Then `verifyAnswer()`:

1. Resolves each citation's policy by id, falling back to a normalised title match.
2. Looks the quote up in that policy's stored text.
3. Drops anything that fails, into a `rejected` list with a reason.
4. De-duplicates repeated quotes.
5. Sets `unsupported` when the answer made claims but **none** could be traced to a policy.

Matching is whitespace- and case-insensitive but **keeps punctuation** — PDF extraction introduces line breaks and runs of spaces that don't exist in the sentence a reader sees, so an exact match would reject honest quotes, while dropping punctuation would let meaningfully different text match. Quotes shorter than 12 characters are never accepted: they match by accident and are evidence of nothing.

> **WHY DISCARD RATHER THAN FLAG**
>
> A citation nobody can follow is worse than no citation, because it looks authoritative. The same check runs in the Edge Function *and* again in the browser.

Saying "not covered". The model returns a `covered` flag and, when false, questions the documents *can* answer. That flag is optional in the type — a browser may be talking to an older hand-deployed function that never sends it, and a missing flag therefore means "work it out from the prose", not "not covered".

Chats. Nothing is written until an answer actually arrives, so a failed question leaves no half-conversation behind.

09 PDFs and form filling

CONCERN	LIBRARY	WHERE
Writing a camp plan / menu	`jspdf` + `jspdf-autotable`	`lib/campPlanPdf.ts`, `lib/menuPdf.ts`
Filling an existing fillable PDF	`pdf-lib`	`lib/formFill.ts`
Reading text out of a PDF	`pdfjs-dist`	`lib/documentText.ts`

The camp plan assembles the event, its site (contact details included), the menu days, the packing list, the food plan and the shopping list into one printable document. Filenames are slugged: `camp-plan—<slug>-<start-date>.pdf`.

Form filling. Upload an official fillable PDF once. BaseMind reads its field names and you map each to a data key — `event.date_range`, `hospital.phone`, `safety.leader_in_charge`, `safety.full_text` (the whole rendered plan), `today`. The mapping lives in `camp_forms.field_map`.

> **MAINTENANCE NOTE**
>
> These keys are stable strings stored in the database. Renaming one orphans every existing mapping. Add new keys rather than changing old ones.

---

10 Person icons

Everyone gets a camp badge — one of sixteen emblems in one of eight colours, 128 combinations — shown in the header and anywhere people are listed. It is **computed from the user's id in the browser**, which is the whole point:

- a new account has one immediately, with nothing to upload and nothing to set up
- it is identical on every device
- it still draws with no connection
- there is no state in which somebody has no icon

Your profile → Your icon lets you pin an emblem and colour instead, or switch to your initials. Two nullable columns store that choice; "Start again" clears them and hands the choice back to your id. No image is uploaded, generated or hosted anywhere.

11 Feature flags

Nine switches, one row each. Rows rather than a single settings blob, so two admins toggling different things cannot overwrite each other.

KEY	LABEL	GATES
`equipment`	Equipment	The Inventory tab (**not** the `/equipment` route)
`loans`	Borrowing	`/loans`
`events`	Calendar	`/events`, `/events/new`, `/events/:id`
`sites`	Sites	`/sites`
`menus`	Menus	`/menus`
`food`	Food	`/food`
`forms`	Forms	`/forms`
`safety`	Safety plans	The safety plan section on the camp page
`policies`	Policies	`/assistant` and `/policies`

Changes propagate live over Realtime — no redeploy, no refresh. Turning something off hides its page and menu item for everyone; the data stays exactly where it is.

12 Setting it up from scratch

PREREQUISITES

- Node.js 20 or newer, and npm

- A Supabase account
- Optionally, a Google AI Studio key for the two AI features

1 · Install

```
cd basemind-app
npm install
```

2 · Create a Supabase project

From **Project Settings** → **API**, note the **Project URL** and the **anon / public key**. The anon key is meant to be public — RLS is what protects the data, not key secrecy.

3 · Run the migrations

Open the SQL Editor and run everything in `supabase/migrations/` **in order, once each**.

MIGRATION	ADDS
`0001_init.sql`	`users` and `equipment`, the signup trigger, the RLS helpers
`0002_fix_admin_bootstrap.sql`	Nothing on a fresh project — see the note inside the file
`0003_camps_and_food.sql`	Camping sites, events, packing lists, food store, camp food plans
`0004_equipment_loans.sql`	Borrowing
`0005_menus.sql`	Menus, meals, ingredients, event headcount, the two menu functions
`0006_safety.sql`	Hospital/emergency fields per site, safety plans and scenarios
`0007_camp_forms.sql`	Uploaded fillable PDFs, field mappings, the `camp-forms` bucket
`0008_feature_flags.sql`	The eight original feature switches
`0009_policies.sql`	Policy documents, extracted text, the `policies` bucket, the ninth switch
`0010_user_icons.sql`	The emblem/colour columns; removes the old `avatars` bucket
`0011_assistant_chats.sql`	Saved chats and messages, policy source URLs, unique titles

THE REPO README IS BEHIND

`basemind-app/README.md` lists migrations only as far as `0010`. `0011` exists and is required for the assistant's saved chats and for adding policies by link.

`0002` and `0010` are safe to run twice; the rest are not. The calendar, sites and food pages will error until `0003` has been applied.

4 · Environment variables

```
cp .env.example .env.local
```

```
VITE_SUPABASE_URL=https://<your-project>.supabase.co
VITE_SUPABASE_ANON_KEY=<your anon key>
```

Both are required. Missing either replaces the app with a screen naming what's absent.

5 · Run it

```
npm run dev
```

Vite serves on port 5173 with `--host`, so it's reachable from a phone on the same network.

6 · Decide on email confirmation

Supabase turns **Confirm email** on by default. This app already gates access behind admin approval, so that step is mostly redundant friction for a small group. To turn it off: **Authentication** → **Providers** → **Email** → toggle off **Confirm email**. To confirm one manually:

```
update auth.users set email_confirmed_at = now() where email = 'you@example.com';
```

7 · Create the first Admin

There is no admin yet to approve the first signup, so:

1. Go to `/register` and create an account. You'll land on the pending screen.
2. In the SQL Editor, with your own email, run the statement below.
3. Sign out and back in. You'll land on Equipment with the admin gear in the header.

```
update public.users set role = 'Admin', status = 'Active' where email = 'you@example
```

This touches `public.users` only — it does **not** confirm the email address. That's the separate `auth.users` step above.

8 · Optional: sample data

Run `supabase/seed/dummy_data.sql` to fill the app with fictional equipment across four rooms and six categories, four camping sites, a food store, and five events — two with full packing lists and food plans. Every row is prefixed `[sample]`, and `remove_dummy_data.sql` removes all of it without touching anything you added. *Every screenshot in this document is of exactly this seed data.*

9 · Optional: the AI features

For each of the three functions in `supabase/functions/`: **Supabase dashboard → Edge Functions → create a function with that exact name → paste the file's contents.** `safety-plan` and `policy-chat` need a `GEMINI_API_KEY` secret; `fetch-policy-pdf` needs none. Skip any of them and the feature degrades to its non-AI path.

---

13 Deploying to Vercel

1. **Push to GitHub.** Create the empty repo first, then push.
2. **Import into Vercel.** `vercel.json` sets the build command, the output directory, and — importantly — a rewrite that serves `index.html` for every path. Without it, a client-side route like `/events/abc` 404s when someone reloads or opens a deep link.
3. **Set environment variables.** `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`. These are read at **build** time, so redeploy after adding them if the first deploy

already ran.

4. **Supabase Auth redirect URLs.** If email confirmation is on, add your Vercel domain under **Authentication → URL Configuration → Redirect URLs**, or confirmation links bounce back to `localhost`.

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

> **IF THE REPO ROOT IS THE PARENT FOLDER**
>
> Set Vercel's **Root Directory** to `basemind-app` so it finds `vercel.json` and `package.json`.

Order matters: get the Supabase project and schema done first, since the build needs those variables.

---

14 Verifying it works

- [ ] `npm run dev`

boots with no console errors and the purple theme applies.

- [ ] `npm run build`

com
—

```
tsc --noEmit
```

it runs

```
```

first, so it fails on any type error.

- [ ] Register a second test account, approve it as each role in turn, and confirm each sees exactly the controls its role allows.
- [ ] A Viewer or Pending account cannot reach `/users` or `/admin`.
- [ ] **The real test:**

with a Viewer session, make a direct write to the

```
equipment
```

Sup
RES

endpoint, bypassing the UI, and confirm Postgres rejects it.

- [ ] Two sessions as different Active users: edit equipment in one tab and watch it appear in the other without a refresh.
- [ ] An Admin rejects a user → that account is immediately locked out.
- [ ] Turn a feature off in Admin and watch the section disappear in the other tab, live.
- [ ] Go offline in devtools, add an item, come back online, and watch the sync toast.

15 Troubleshooting

SYMPTOM	CAUSE AND FIX
"Configuration missing" screen	Env vars unset. Locally, fill `.env.local`; on Vercel, add them and **redeploy** — they're read at build time.
`Email not confirmed` on sign-in	Supabase's Confirm email is on. Click the link, turn it off, or set `email_confirmed_at` manually.
Stuck on "pending approval"	No Admin has approved you. For the *first* account, run the bootstrap SQL in step 7.
Bootstrap SQL runs but changes nothing	The pre-0002 trigger silently reverting `role` / `status` for callers with no JWT. Run `0002_fix_admin_bootstrap.sql`.
Calendar / Sites / Food pages error	Migration `0003` hasn't been run.
Admin says "No feature switches found"	Migration `0008` hasn't been run. Everything stays on until it is.
Assistant errors on every question	`policy-chat` not deployed, or `GEMINI_API_KEY` not set. Documents still work.
Gemini 404 naming the model	`GEMINI_MODEL` isn't available on your key. The error lists the models that are.
Editing an Edge Function had no effect	They don't deploy from git. Paste the file into the dashboard again.
"No text could be read from that PDF"	It's a scan or photograph — the pages are images. Export a text-based PDF from the original.
Deep link 404s on Vercel	The `rewrites` rule isn't being picked up — check Vercel's Root Directory setting.
`Failed to fetch dynamically imported module`	A deploy landed mid-session. The app reloads itself once automatically; seeing the error means that reload already happened.

SYMPTOM	CAUSE AND FIX
Edit/delete buttons missing but add works	You're offline. `canEdit` and `canDelete` require a connection by design.

16 Working on the code

```
npm run dev       # Vite dev server on :5173, exposed on the LAN
npm run build     # tsc --noEmit, then vite build → dist/
npm run preview   # serve the built output
```

There is no linter, formatter or test suite configured. `npm run build` is the gate — it type-checks the whole project before bundling.

CONVENTIONS WORTH MATCHING

- **Comments explain *why*, not *what*.** The existing comments are unusually good about naming the failure a piece of code exists to prevent. Match that.
- **Business logic goes in `src/lib/`**, framework-free and independently readable. Components compose it.
- **Read through `useRealtimeTable`, write straight to Supabase.** Don't update local state after a mutation — let the realtime event do it.
- **Row types are `type` aliases**, never `interface`.
- **Never edit a migration that's already been run.** Migration 0002's own comment calls out exactly this mistake having been made once already.
- **Path alias: `@/` → `src/`.**

17 Known gaps and things to watch

Honest notes, not criticisms — most are already acknowledged in the source.

1. **The `equipment` feature flag doesn't gate its route.** Every other flag wraps its routes in `RequireFeature`; `/equipment` doesn't, and `/` still redirects there.
2. **The app README stops at migration 0010.** `0011` exists and is required. Its closing "What's deliberately not here" section is also well out of date — it still says camp planning, documents and the chatbot are cut, and all three have since been built.
3. **Edge Functions are deployed by hand.** There's no deploy step in any workflow, so the code in the repo and the code actually running can drift. The optional `covered` field exists precisely to tolerate that.
4. **DNS rebinding isn't covered** by `fetch-policy-pdf`'s SSRF guards, as its own comments state. Literal private addresses and metadata hostnames are blocked.
5. **No tests.** `tsc --noEmit` is the only automated check. The pure-logic modules in `src/lib/` are written as pure functions and would be straightforward to test — the obvious first place to start.
6. **A stale cached role widens the UI, not access.** Worth restating because it looks alarming and isn't: RLS still runs server-side on every write.
7. **`0001_init.sql` was edited after being run.** Acknowledged in 0002's comments. A database created from the original `0001` needs `0002`; one from the current `0001` doesn't, and running it anyway is harmless.

Written against commit `3cb1629` on `master` and verified against a live instance on 27 August 2026. Every screenshot is a real capture of the running app against the repository's own `[sample]` seed data. A long-form Markdown version lives at `DOCUMENTATION.md` in the repo, with the full-resolution images in `docs/screenshots/`.